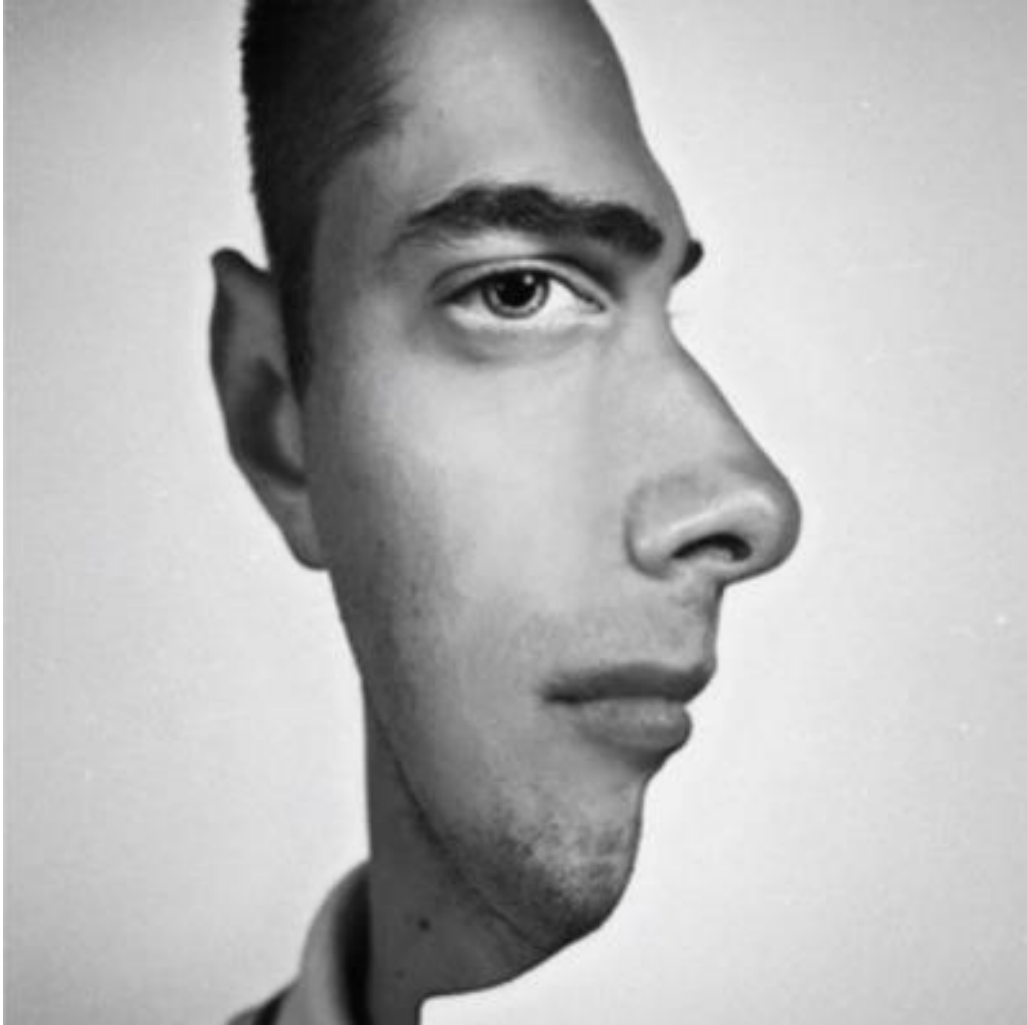


# Convolutional Neural Networks

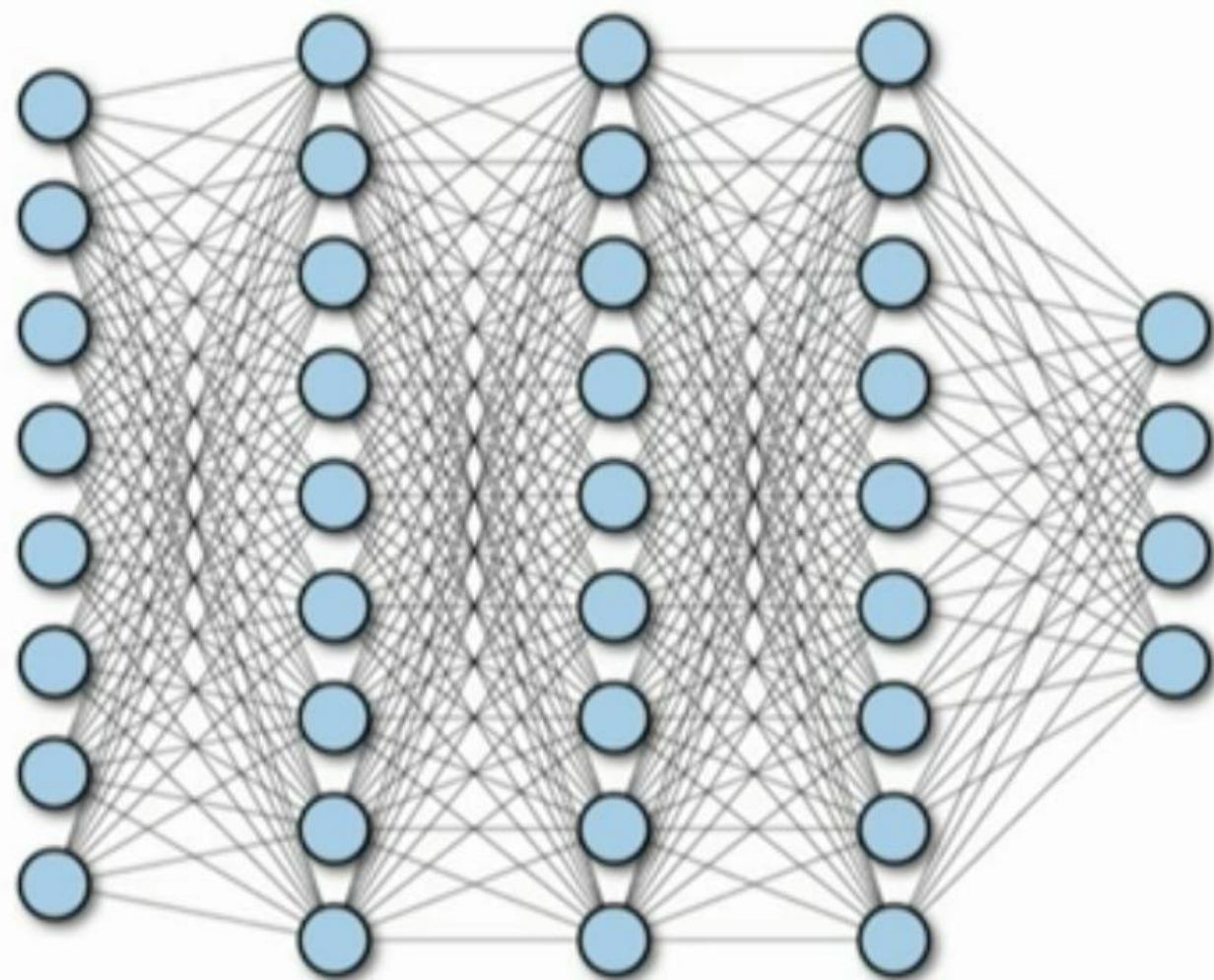
Adapted from [IntroToDeepLearning.com](http://IntroToDeepLearning.com) MIT

- How do we perceive images?



Source: <https://towardsdatascience.com>

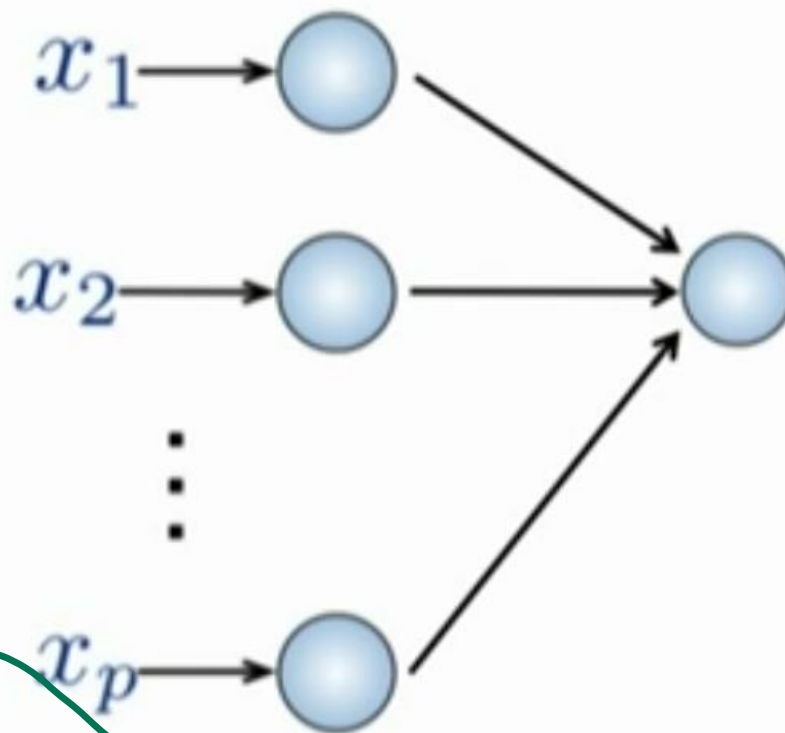
# Fully Connected Neural Network



# Fully Connected Neural Network

## Input:

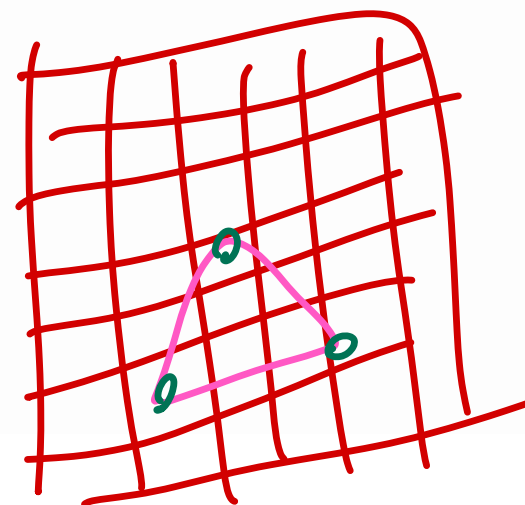
- 2D image
- Vector of pixel values



## Fully Connected:

- Connect neuron in hidden layer to all neurons in input layer
- No spatial information!
- And many, many parameters!

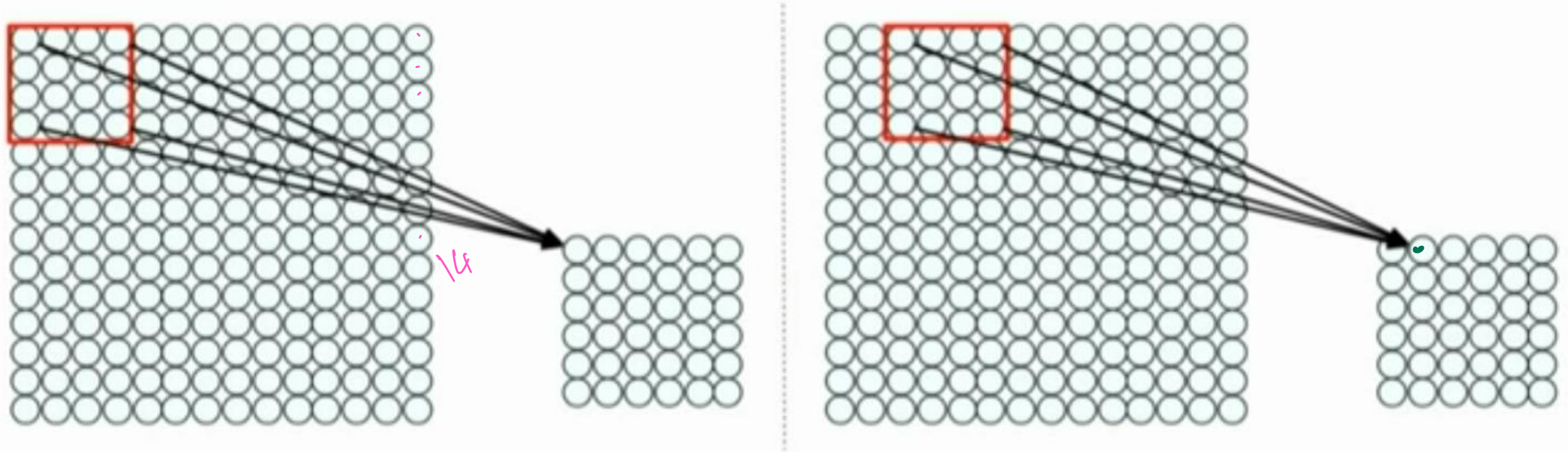
① Complex  
• Loss spatial orientation







# Using Spatial Structure



Connect patch in input layer to a single neuron in subsequent layer.  
Use a sliding window to define connections.  
How can we **weight** the patch to detect particular features?

$n \times n$  is image size

$f \times f$  is filter size

$s=1$  (stride = 1)

$\Rightarrow$  output  $(n-f+1) \times (n-f+1)$

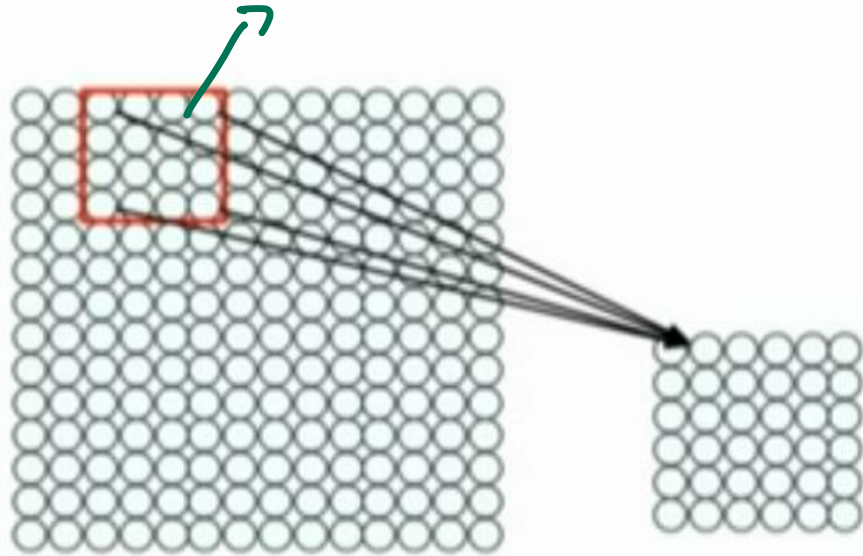
$s > 1$

Output:

$$\left( \frac{n-f+s}{s} \right) \times \left( \frac{n-f+s}{s} \right)$$



# Feature Extraction with Convolution

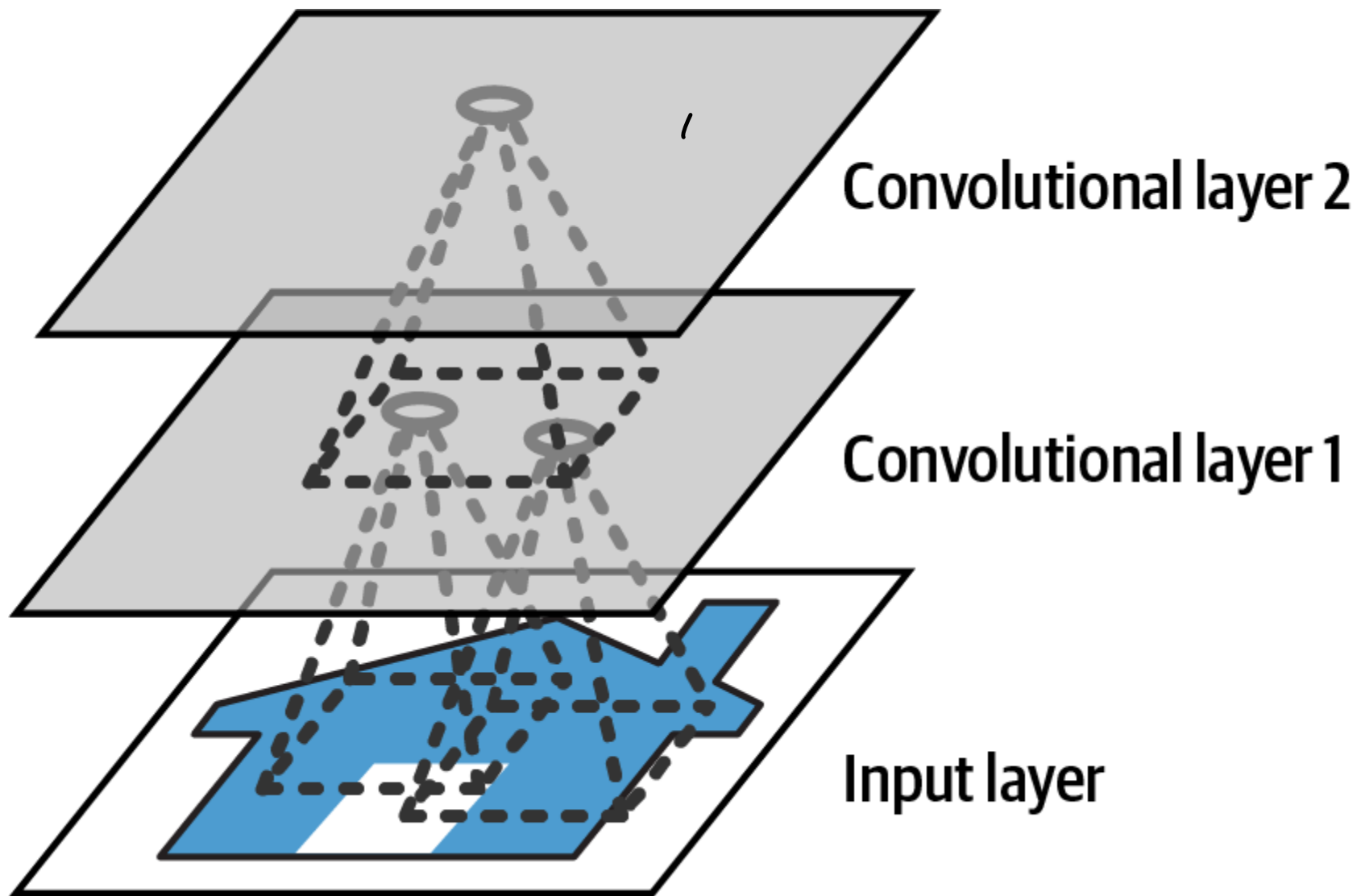


- Filter of size 4x4 : 16 different weights
- Apply this same filter to 4x4 patches in input
- Shift by 2 pixels for next patch

This “patchy” operation is **convolution**

- 1) Apply a set of weights – a filter – to extract **local features**
- 2) Use **multiple filters** to extract different features
- 3) **Spatially share** parameters of each filter





X or X?

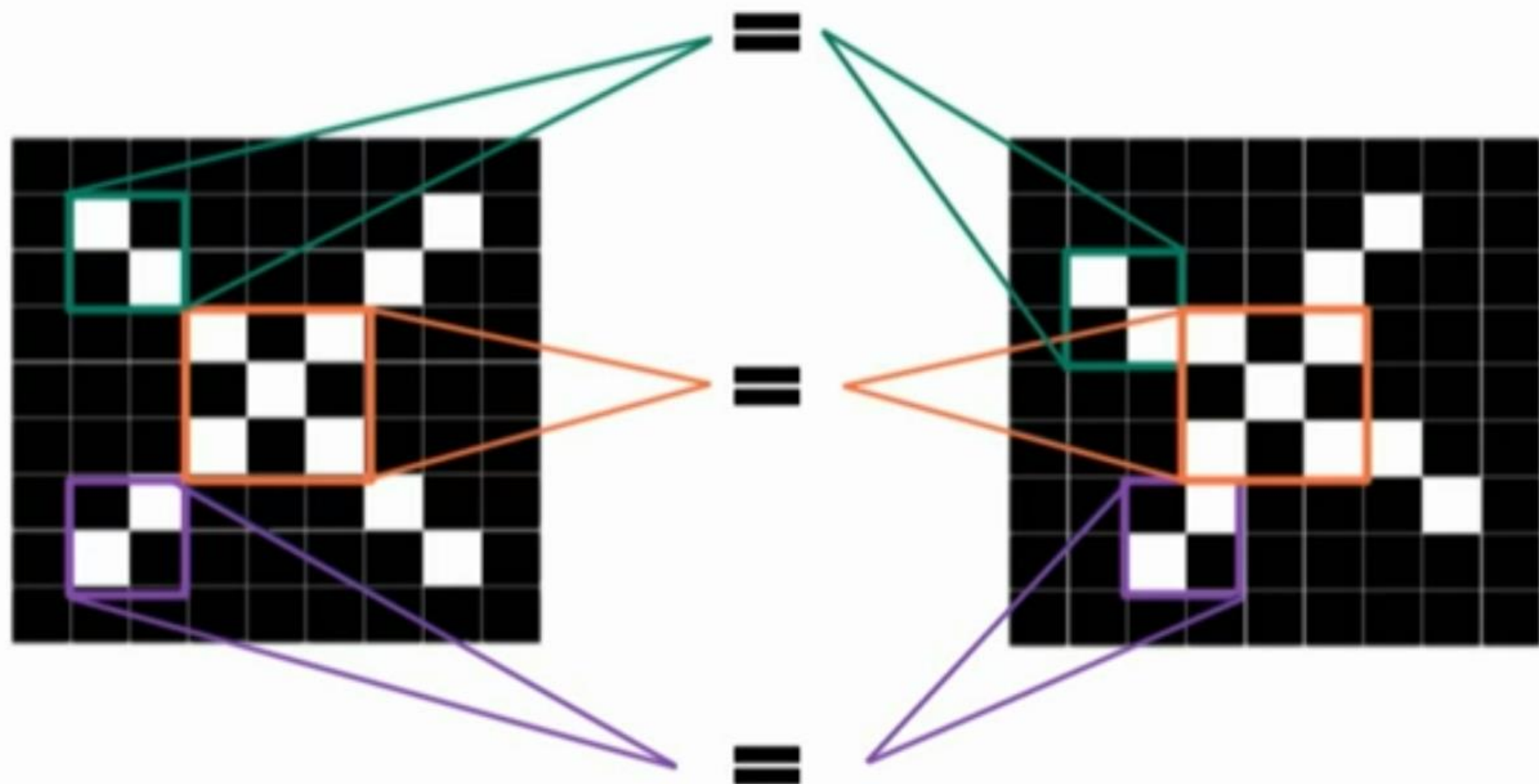
?

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |
| -1 | 1  | -1 | -1 | -1 | -1 | -1 | 1  | -1 |
| -1 | -1 | 1  | -1 | -1 | -1 | 1  | -1 | -1 |
| -1 | -1 | -1 | 1  | -1 | 1  | -1 | -1 | -1 |
| -1 | -1 | -1 | -1 | 1  | -1 | -1 | -1 | -1 |
| -1 | -1 | -1 | 1  | -1 | 1  | -1 | -1 | -1 |
| -1 | -1 | 1  | -1 | -1 | -1 | 1  | -1 | -1 |
| -1 | 1  | -1 | -1 | -1 | -1 | -1 | 1  | -1 |
| -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |
| -1 | -1 | -1 | -1 | -1 | -1 | 1  | -1 | -1 |
| -1 | 1  | -1 | -1 | -1 | 1  | -1 | -1 | -1 |
| -1 | -1 | 1  | 1  | -1 | 1  | -1 | -1 | -1 |
| -1 | -1 | -1 | -1 | 1  | -1 | -1 | -1 | -1 |
| -1 | -1 | -1 | 1  | -1 | 1  | 1  | -1 | -1 |
| -1 | -1 | -1 | 1  | -1 | -1 | -1 | 1  | -1 |
| -1 | -1 | 1  | -1 | -1 | -1 | -1 | -1 | -1 |
| -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |

Image is represented as matrix of pixel values... and computers are literal!  
We want to be able to classify an X as an X even if it's shifted, shrunk, rotated, deformed.

# Features of X



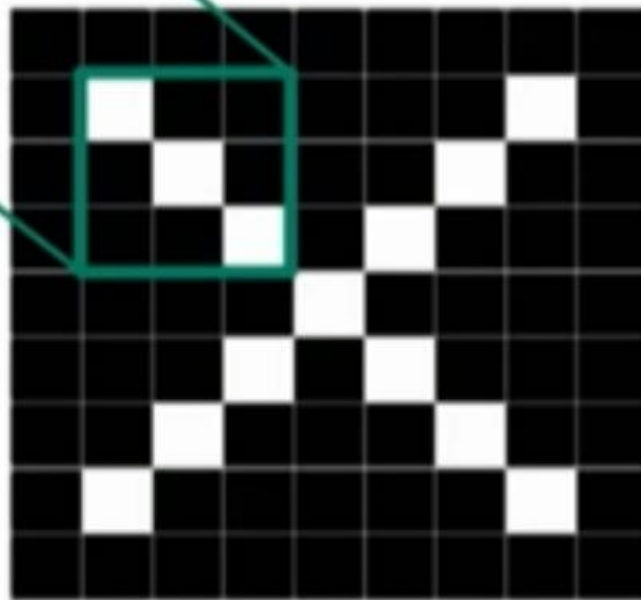
# Filters to Detect X Features

filters

|    |    |    |
|----|----|----|
| 1  | -1 | -1 |
| -1 | 1  | -1 |
| -1 | -1 | 1  |

|               |               |               |
|---------------|---------------|---------------|
| <del>1</del>  | <del>-1</del> | <del>1</del>  |
| <del>-1</del> | <del>1</del>  | <del>-1</del> |
| <del>1</del>  | <del>-1</del> | <del>1</del>  |

|               |               |               |
|---------------|---------------|---------------|
| <del>-1</del> | <del>-1</del> | <del>1</del>  |
| <del>-1</del> | <del>1</del>  | <del>-1</del> |
| <del>1</del>  | <del>-1</del> | <del>-1</del> |



$S=1$

|   |    |   |  |  |  |
|---|----|---|--|--|--|
| 7 | -1 | 1 |  |  |  |
|   | 9  |   |  |  |  |
|   |    |   |  |  |  |
|   |    |   |  |  |  |
|   |    |   |  |  |  |
|   |    |   |  |  |  |

$-1 + 1 + 1 + 1 + 1 + 1$

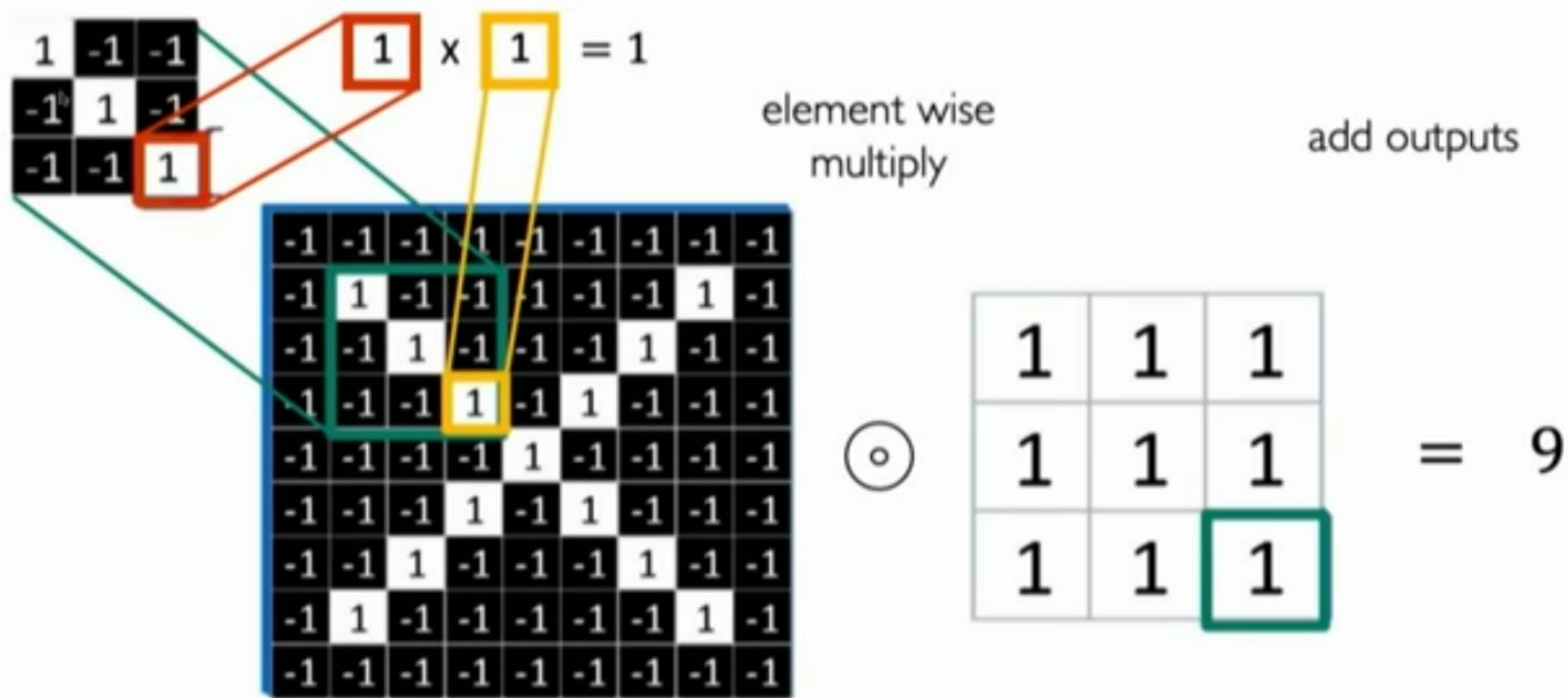
4 :

$S=2$

|   |   |  |
|---|---|--|
| 7 | 1 |  |
|   |   |  |
|   |   |  |
|   |   |  |

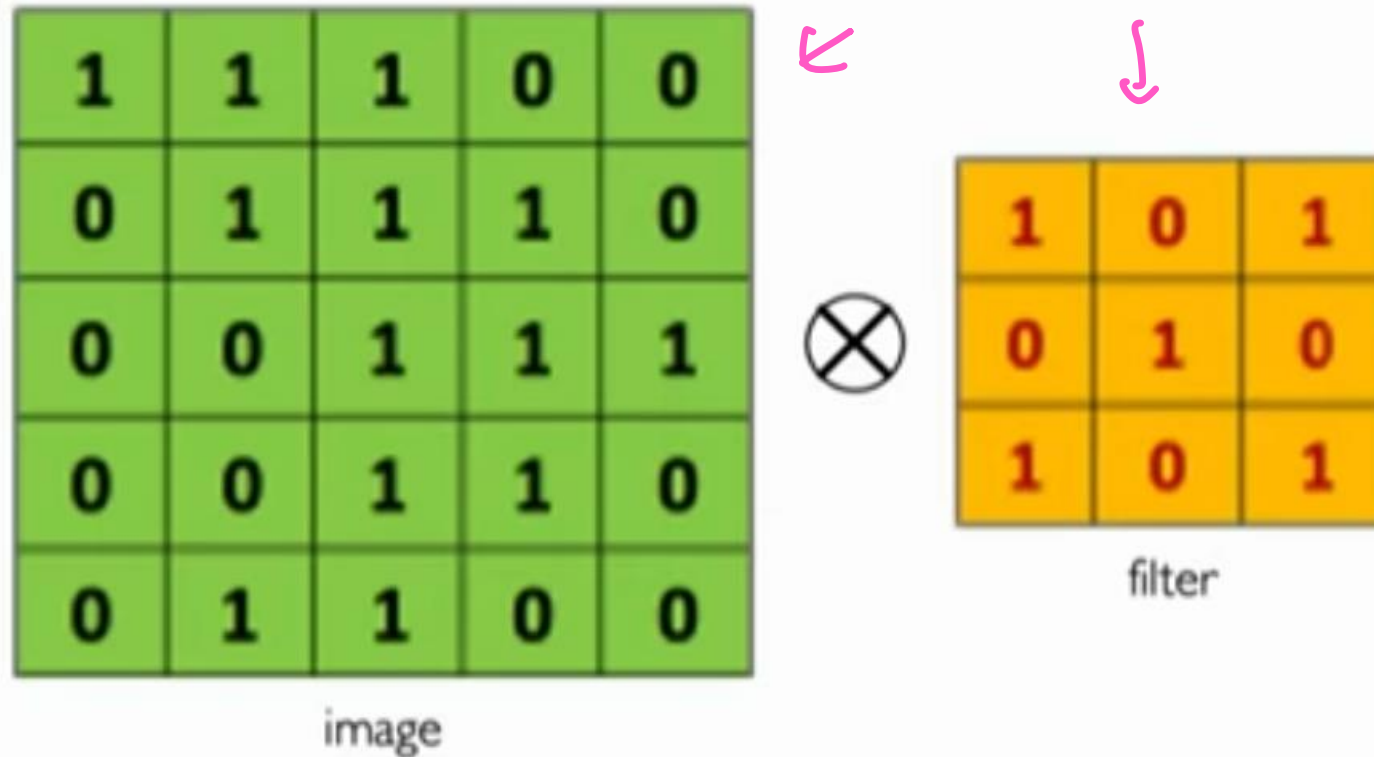


# The Convolution Operation



# The Convolution Operation

Suppose we want to compute the convolution of a 5x5 image and a 3x3 filter:



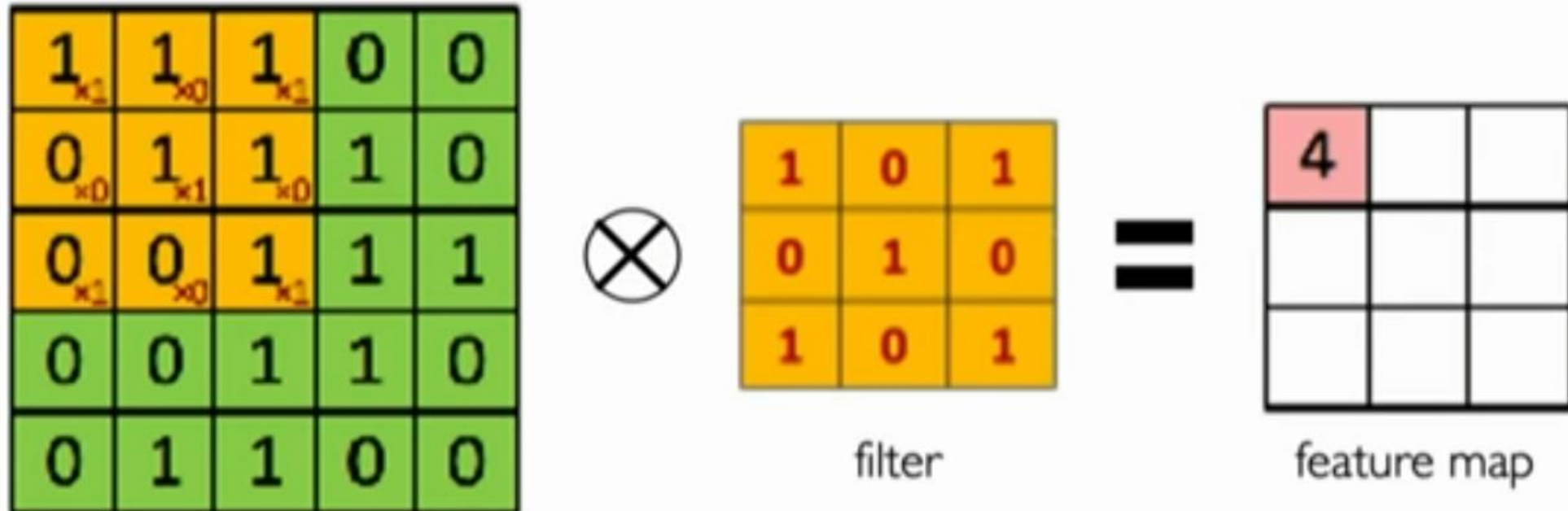
1) dimension of the output  
2) output

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs...

(S=1) (S=2) (S=3) (S=4)

# The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



$$1 + 0 + 1 + 0 + 1 + 0 + 0 + 0 + 1 = 4$$

# The Convolution Operation

$\ast$   $\hookrightarrow$   $V$

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:

Input image (5x5 grid):

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |

Blue arrows indicate the filter's position (rows 2-4, columns 2-4). Small 'x' values indicate element-wise multiplication:  $\times 1$  for green cells and  $\times 0$  for yellow cells.



Filter (3x3 grid):

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |

filter

=

Feature map (3x3 grid):

|   |   |   |
|---|---|---|
| 4 | 3 | 4 |
| 2 | 4 | 3 |
| 2 | 3 | 4 |

Red arrows point to the top row of the feature map. A blue circle highlights the value 4 in the top-left cell.

feature map

$\hookrightarrow$  bias

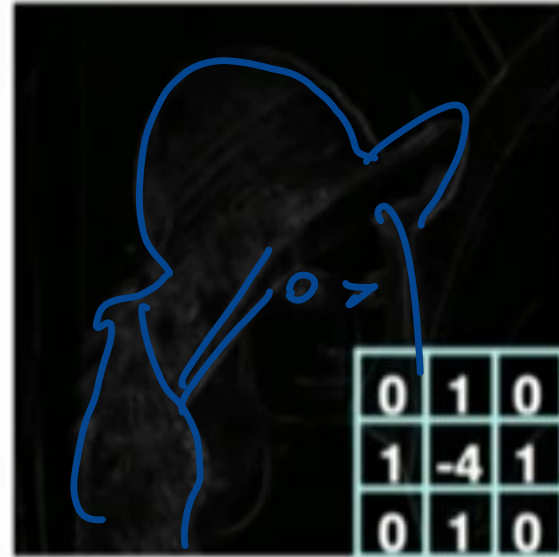
# Producing Feature Maps



Original



Sharpen



Edge Detect

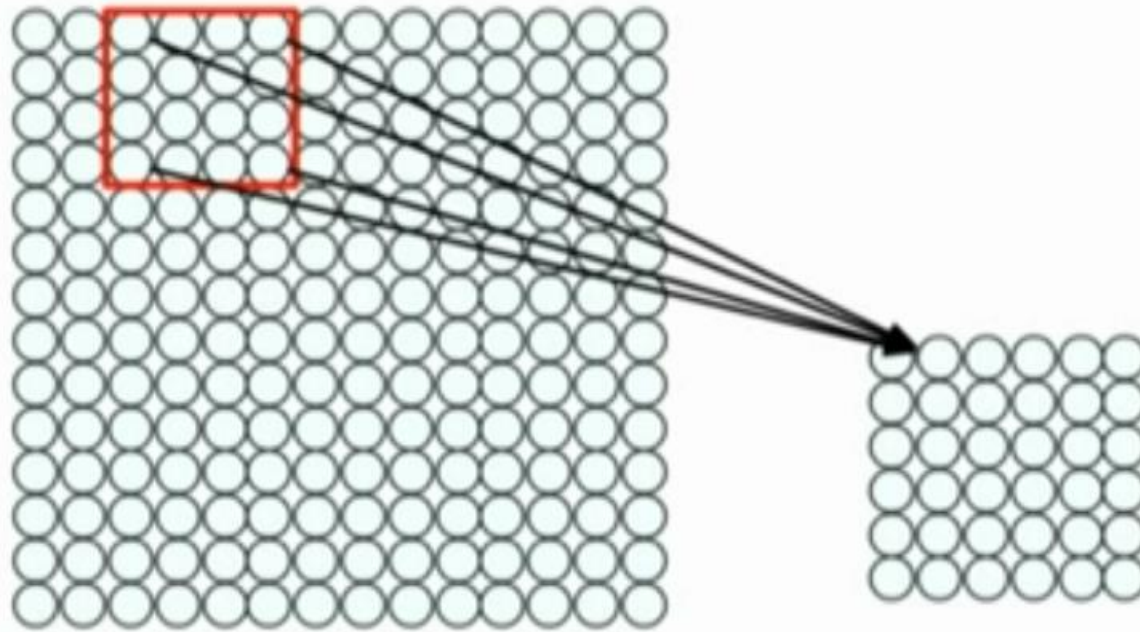


"Strong" Edge Detect



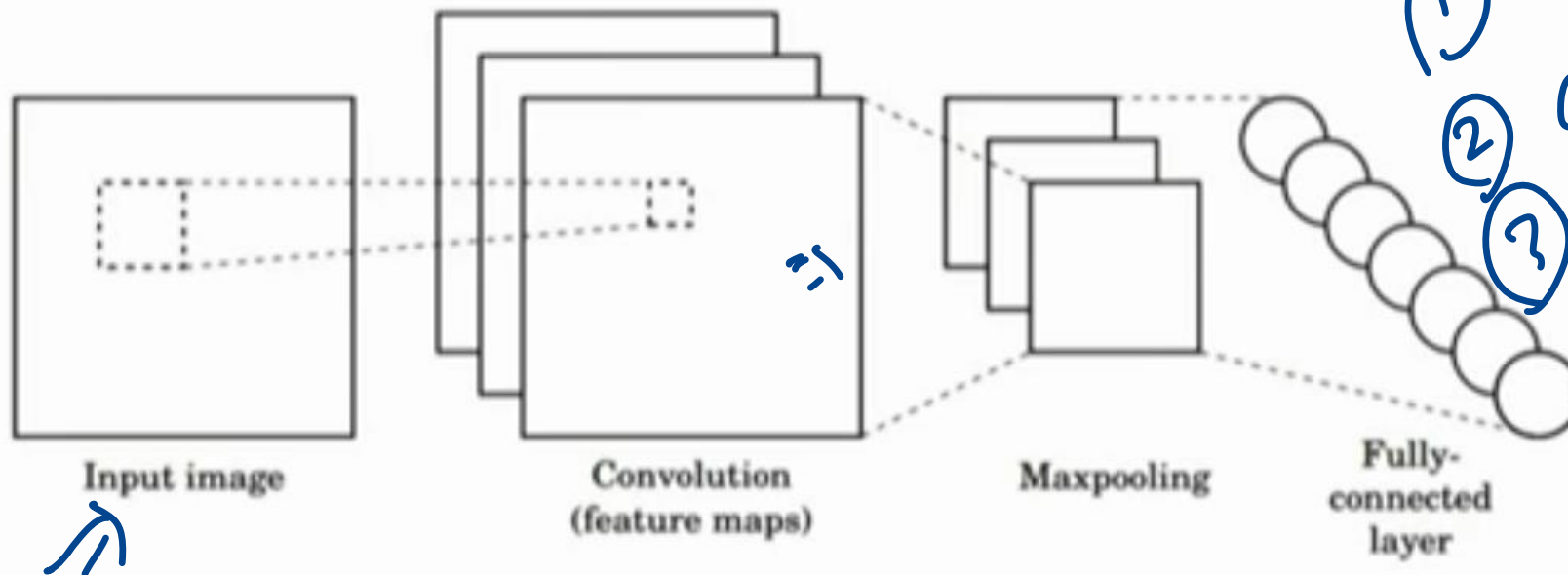


# Feature Extraction with Convolution



- 1) Apply a set of weights – a filter – to extract **local features**
- 2) Use **multiple filters** to extract different features
- 3) **Spatially share** parameters of each filter

# CNNs for Classification



1. **Convolution:** Apply filters to generate feature maps.

2. **Non-linearity:** Often ReLU.

3. **Pooling:** Downsampling operation on each feature map.

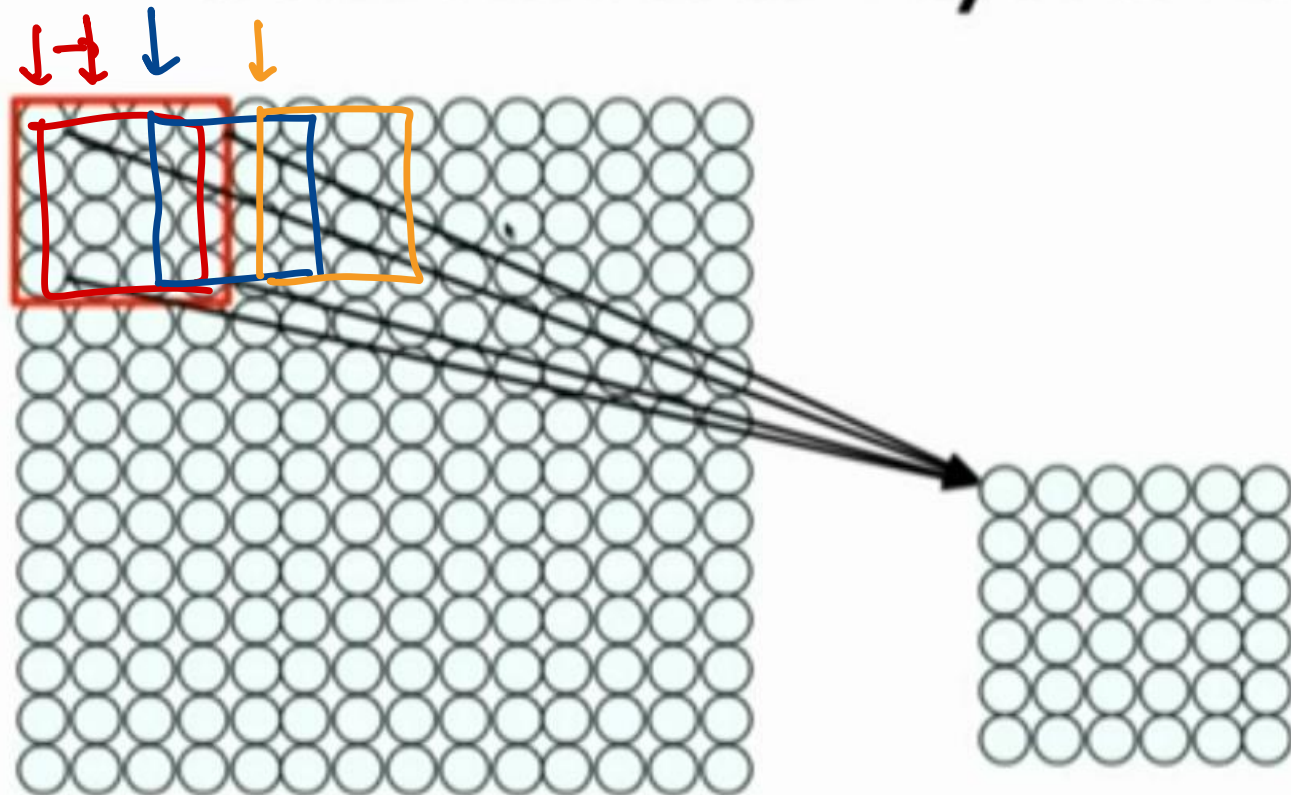
 `tf.keras.layers.Conv2D`

 `tf.keras.activations.*`

 `tf.keras.layers.MaxPool2D`

**Train model with image data.**  
**Learn weights of filters in convolutional layers.**

# Convolutional Layers: Local Connectivity



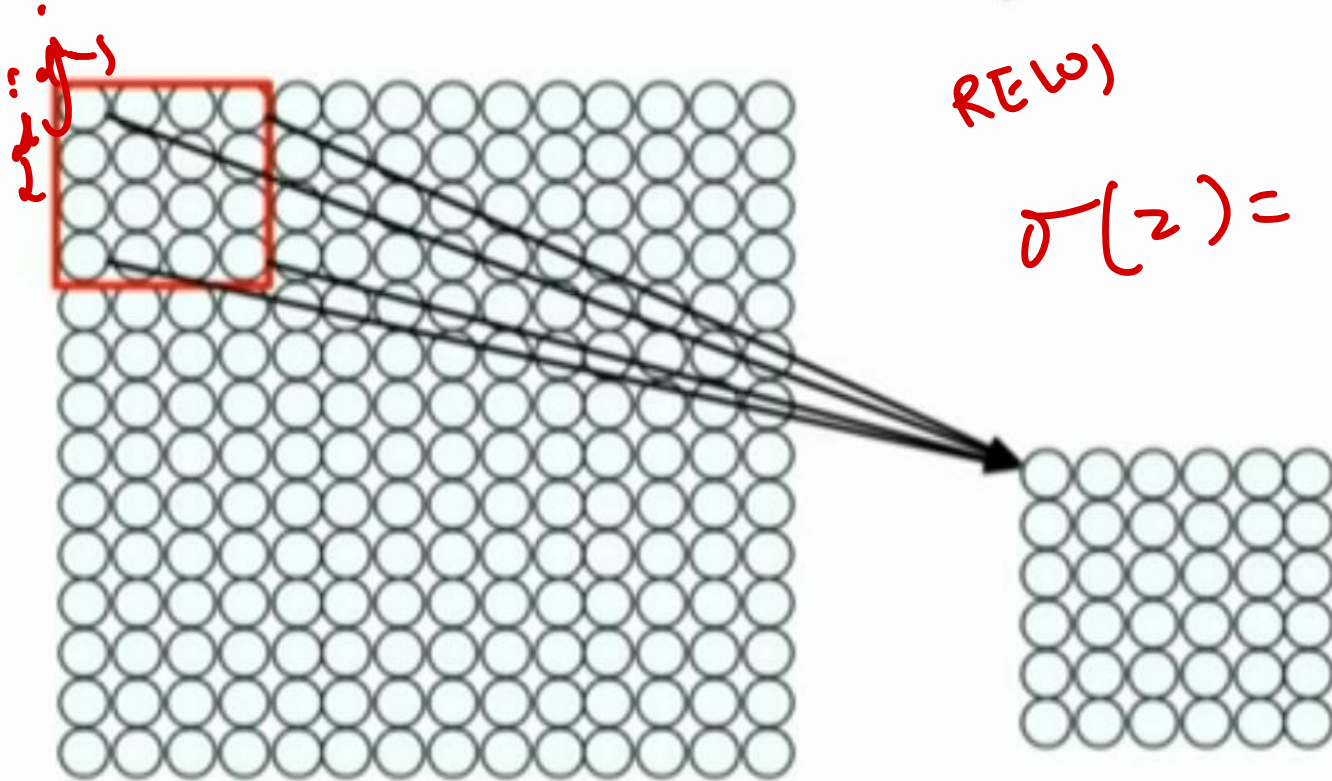
`tf.keras.layers.Conv2D`

**For a neuron in hidden layer:**

- Take inputs from patch
- Compute weighted sum
- Apply bias

# Convolutional Layers: Local Connectivity

 `tf.keras.layers.Conv2D`



4x4 filter: matrix  
of weights  $w_{ij}$

$$\left( \sum_{i=1}^4 \sum_{j=1}^4 w_{ij} x_{i+p, j+q} \right) + b$$

for neuron  $(p,q)$  in hidden layer

$ReLU()$

$$\sigma(z) = \max(0, z)$$

**For a neuron in hidden layer:**

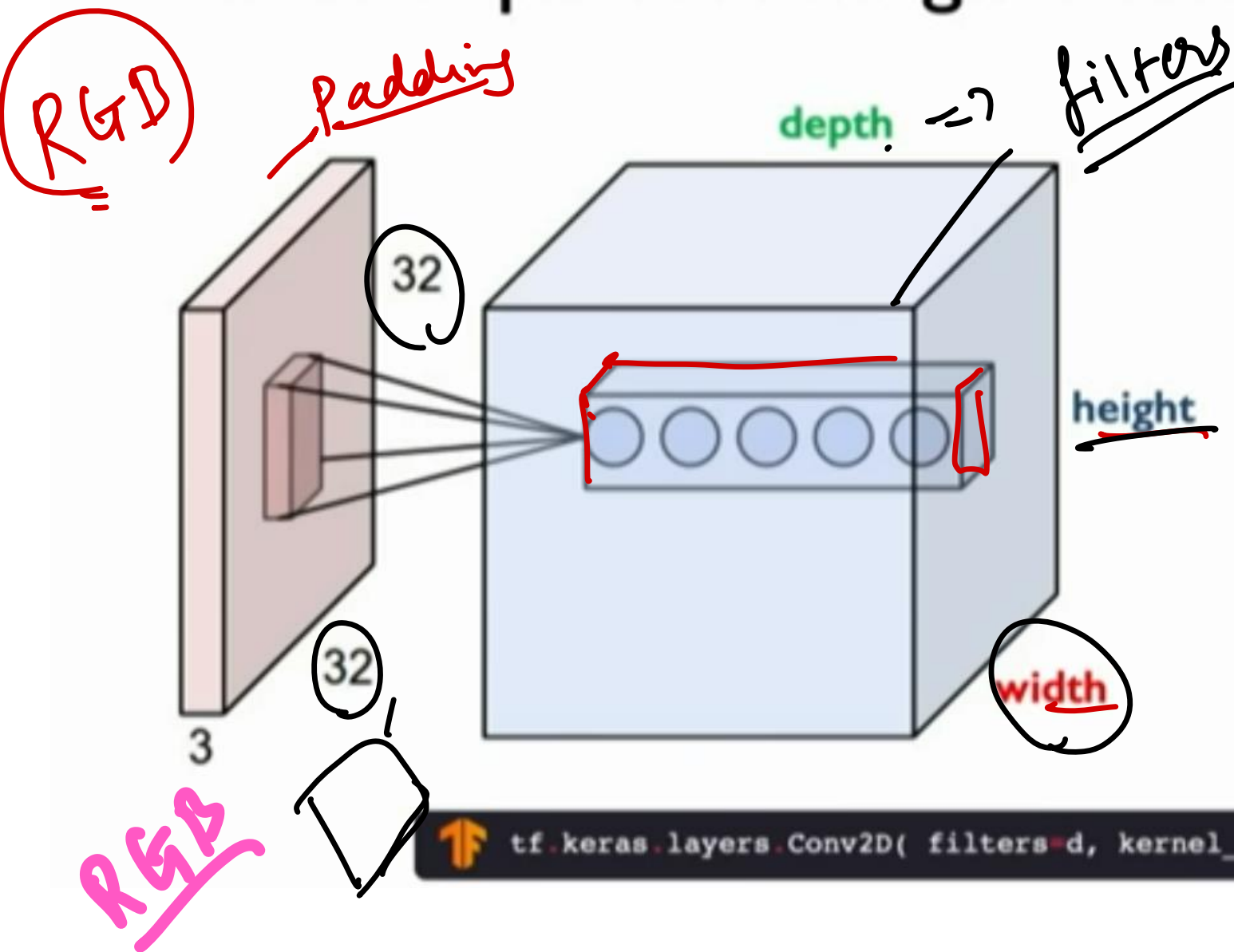
- Take inputs from patch
- Compute weighted sum
- Apply bias

- 1) applying a window of weights
- 2) computing linear combinations
- 3) activating with non-linear function

m



# CNNs: Spatial Arrangement of Output Volume



**Layer Dimensions:**

$h \times w \times d$

where  $h$  and  $w$  are spatial dimensions  
 $d$  (depth) = number of filters

**Stride:**

Filter step size

**Receptive Field:**

Locations in input image that  
a node is path connected to



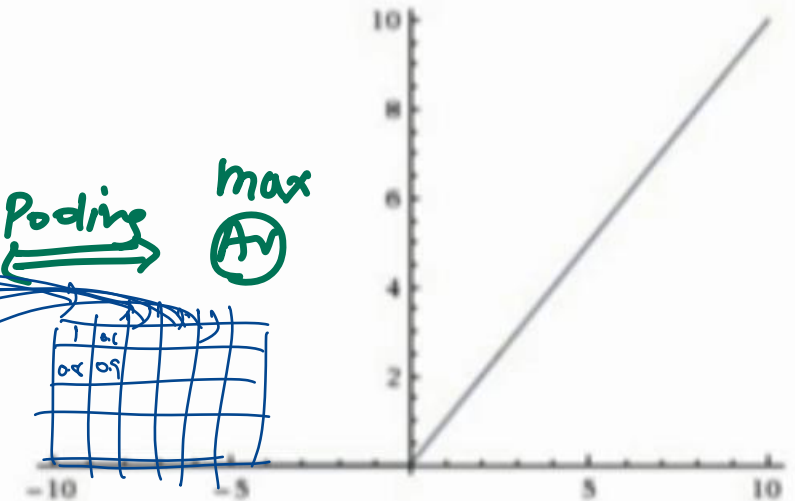
```
tf.keras.layers.Conv2D( filters=d, kernel_size=(h,w), strides=s )
```



# Introducing Non-Linearity

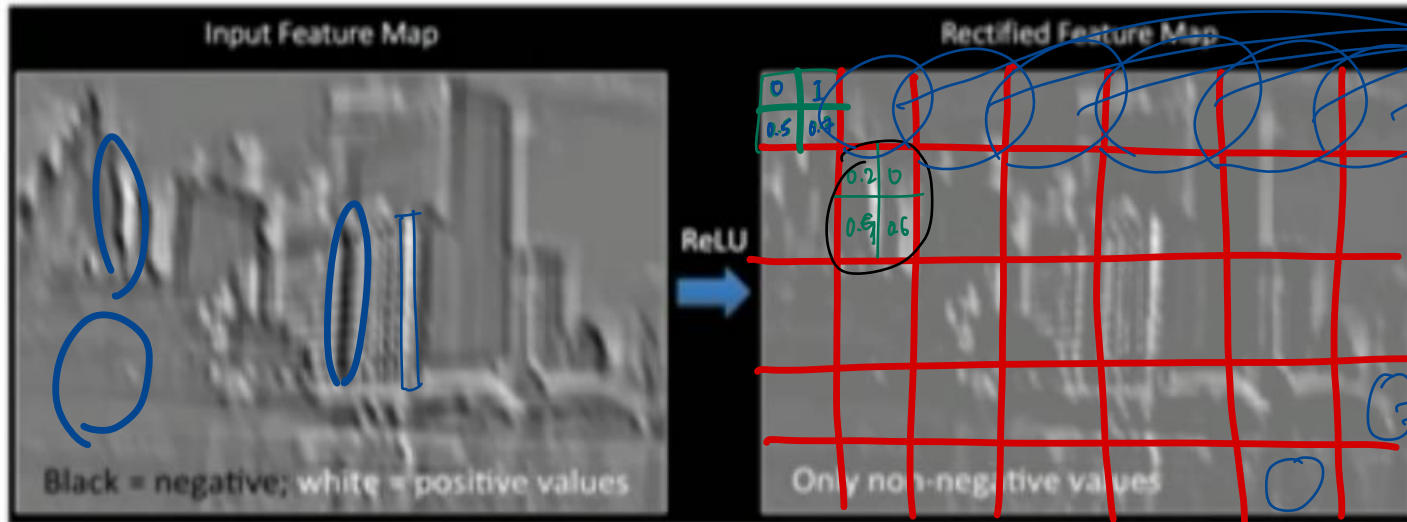
- Apply after every convolution operation (i.e., after convolutional layers)
- ReLU: pixel-by-pixel operation that replaces all negative values by zero. **Non-linear operation!**

## Rectified Linear Unit (ReLU)

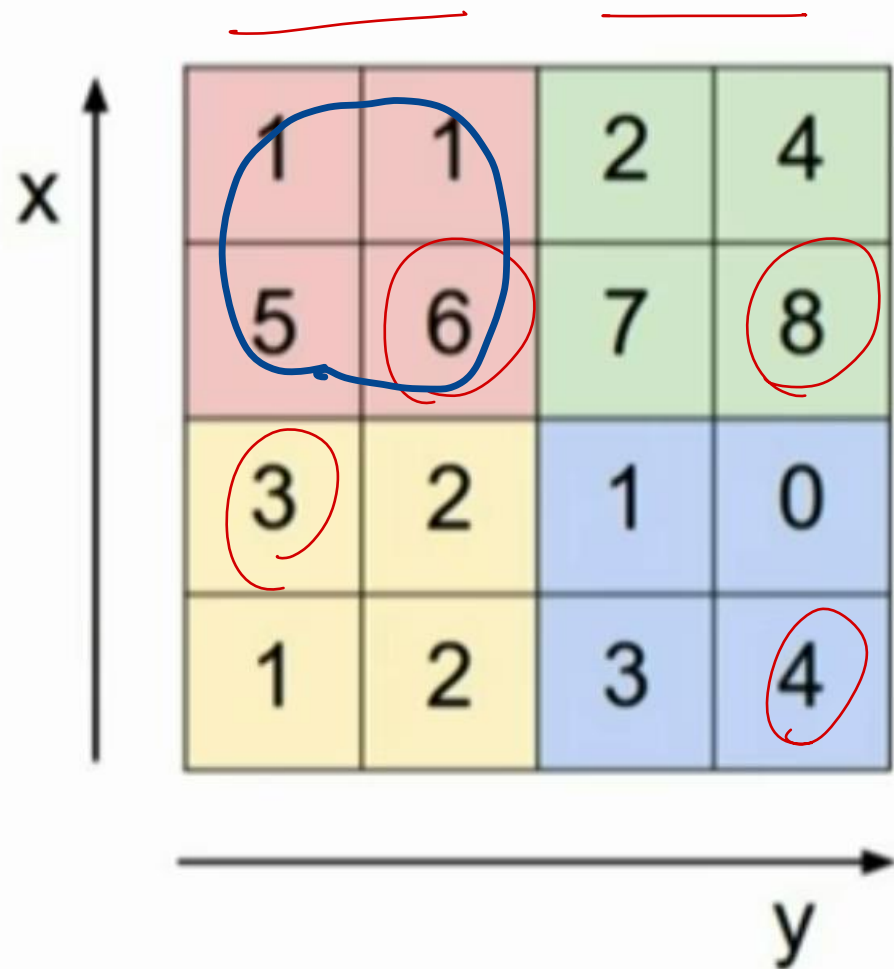


$$g(z) = \max(0, z)$$

 `tf.keras.layers.ReLU`



-1 to 1  
256 0  
Cat



## Pooling

max pool with 2x2 filters  
and stride 2

```
tf.keras.layers.MaxPool2D(  
    pool_size=(2,2),  
    strides=2  
)
```

AvgPool2D

|      |      |
|------|------|
| 3.25 | 5.25 |
| 2    | 2    |

|   |   |
|---|---|
| 6 | 8 |
| 3 | 4 |

- 1) Reduced dimensionality
- 2) Spatial invariance

How else can we downsample and preserve spatial invariance?

Av + Max

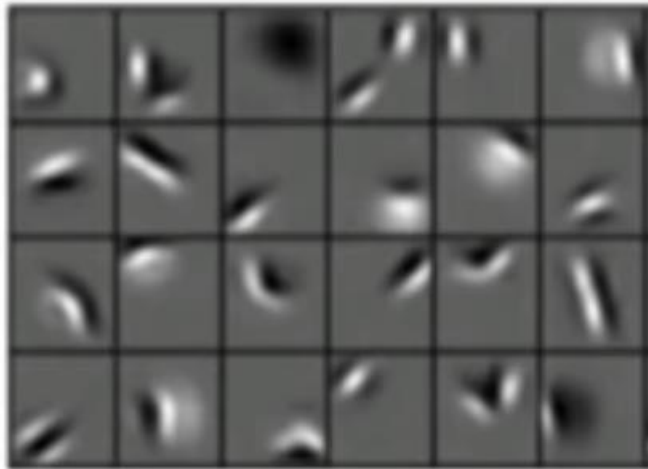
# Representation Learning in Deep CNNs

✓ max pooling

(av.)

max

Low level features



Edges, dark spots

Conv Layer 1



(Mid level features)

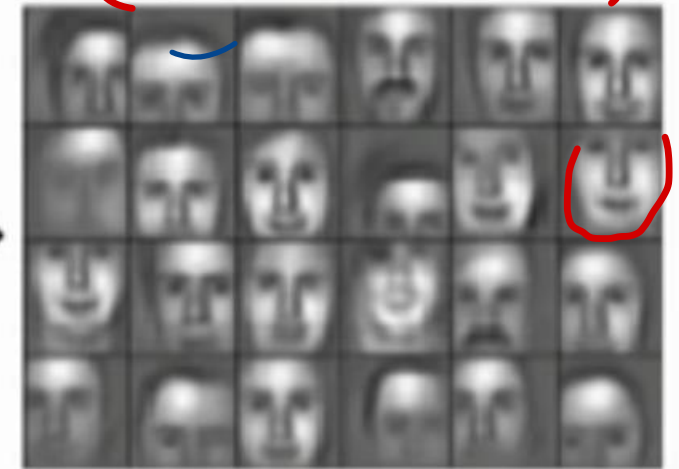


Eyes, ears, nose

Conv Layer 2



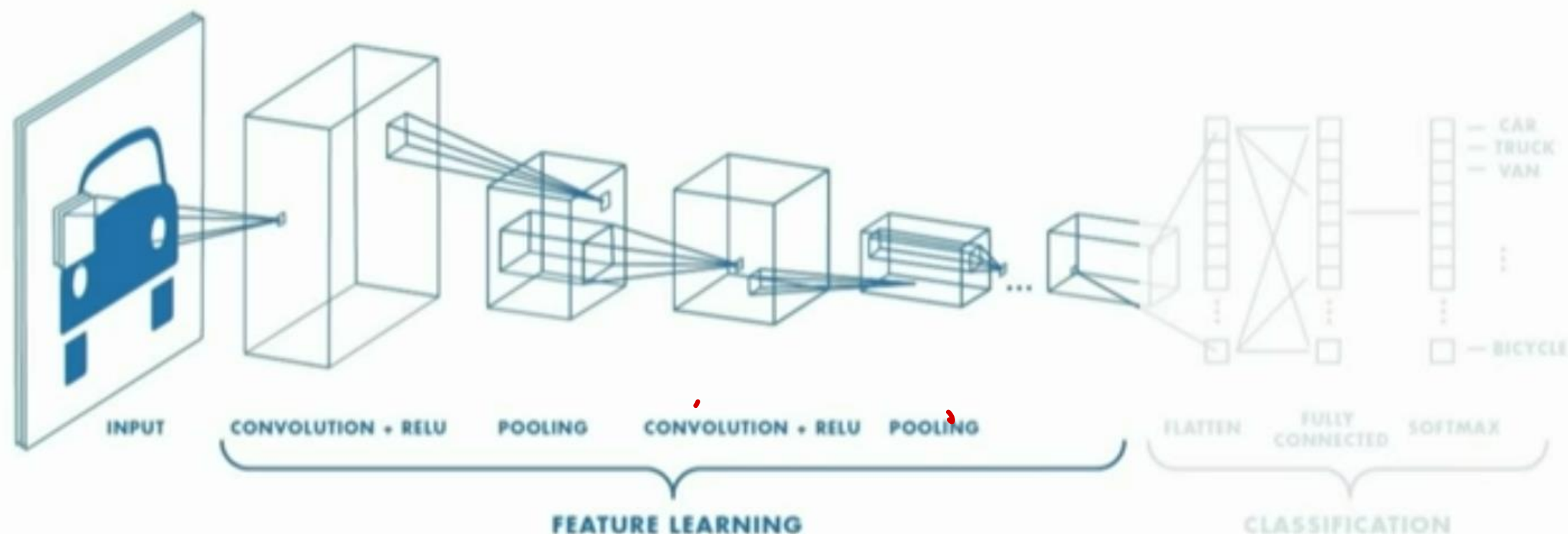
(High level features)



Facial structure

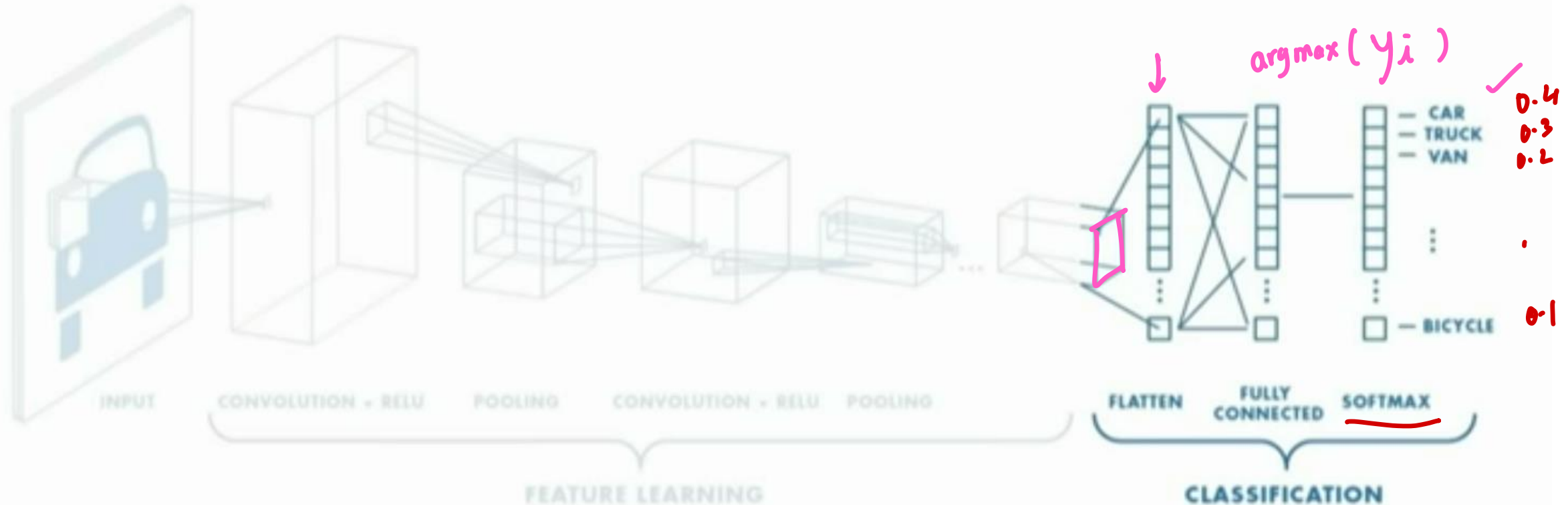
Conv Layer 3

# CNNs for Classification: Feature Learning



1. Learn features in input image through **convolution**
2. Introduce **non-linearity** through activation function (real-world data is non-linear!)
3. Reduce dimensionality and preserve spatial invariance with **pooling**

# CNNs for Classification: Class Probabilities



- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as **probability** of image belonging to a particular class

$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$



CNN cat dog deer

z2 [2, 1, 0.1]

↓  
(softmax)

[0.65, 0.24, 0.11]

↑

20

0.2

(softmax loss function)

$$L = - \sum_{i=1}^K y_i \cdot \log(\hat{y}_i)$$

↓  
labelled  
output

predicted

5

```
import tensorflow as tf

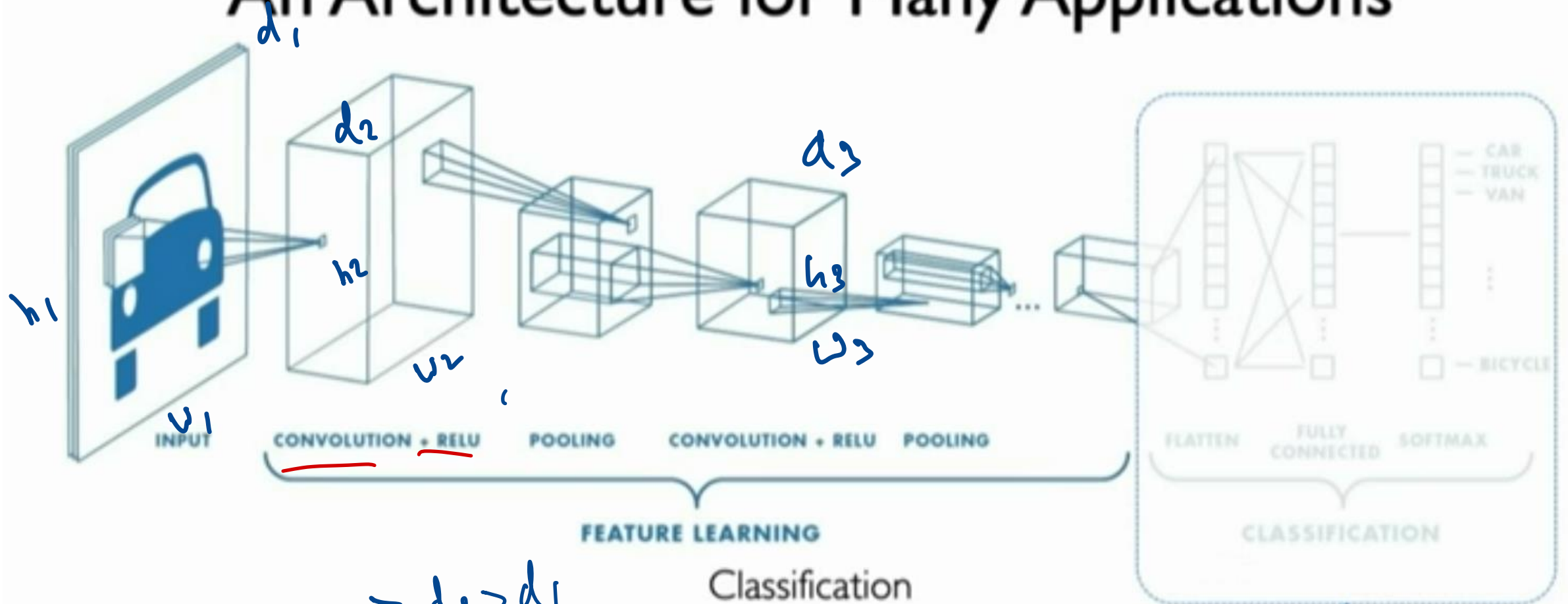
def generate_model():
    model = tf.keras.Sequential([
        # first convolutional layer
        tf.keras.layers.Conv2D(32, filter_size=3, activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

        # second convolutional layer
        tf.keras.layers.Conv2D(64, filter_size=3, activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

        # fully connected classifier
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(1024, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax') # 10 outputs
    ])
    return model
```



# An Architecture for Many Applications



$$d_3 > d_2 > d_1$$

$$h_3 < h_2 < h_1$$

$$w_3 < w_2 < w_1$$

Classification  
Object detection  
Segmentation  
Probabilistic control

# Object Detection

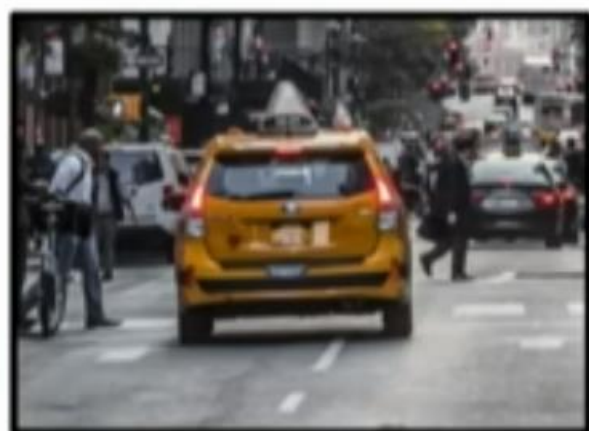


Image  $x$



CNN



Taxi

Class label  $y$

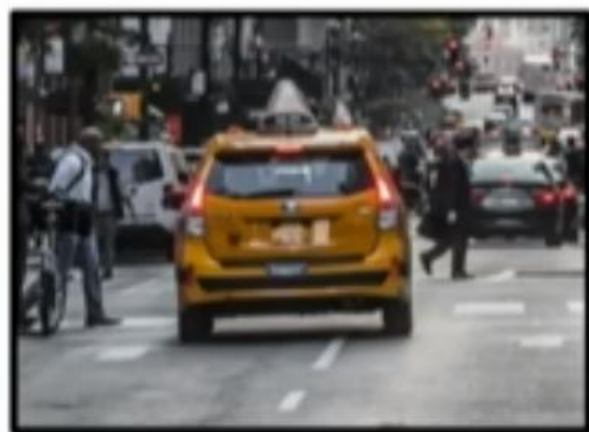


Image  $x$



CNN



Label  $(x, y, w, h)$

- CNN
- CNNs are a special kind of neural network for processing data that has a known, grid-like topology.
- Work extremely well for Image data (2D grid of pixels) and also for time-series data (1D grid taking samples at regular intervals)
- The network employs a mathematical operation, referred to as “convolution”.
- In simple terms, CNNs are neural networks that use convolution in place of general matrix multiplication (in place of at least one of their layers).



- Example – LeNet-5

- Handwritten Character recognition
- 10-categories classification problem

MNIST

A B C D E F G H I J

①

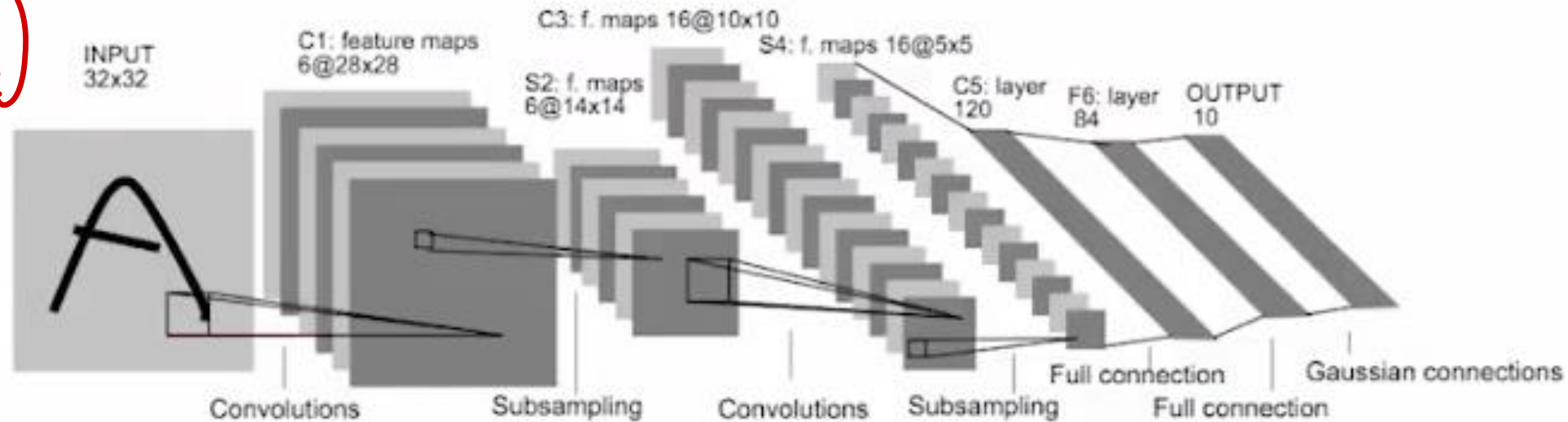


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Courtesy: Yann LeCun

| Layer Out | Type            | Maps | n Size  | K. Size | Stride | Act         |
|-----------|-----------------|------|---------|---------|--------|-------------|
| F6        | Fully connected | —    | 10 / 84 | —       | —      | RBF         |
| ⇒ C5      | Conv.           | 120  | 1x1     | 5x5     | 1      | tanh        |
| S4        | Avg pool.       | 16   | 5x5     | 2x2     | 2      | tanh        |
| ⇒ C3      | Conv            | 16   | 10x10   | 5x5     | 1      | tanh        |
| ⇒ S2      | Avg. Pool       | 6    | 14x14   | 2x2     | 2      | tanh        |
| ⇒ C1      | Conv.           | 6    | 28x28   | 5x5     | 1      | <u>tanh</u> |
| In        | Input           | 1    | 62x32   | —       | —      | —           |

Look from bottom to top

Alex Net

CNN

Search

2012

Alex/Net

650,000  
neurons

17%

60m  
parameters

Exercise:

{ Rebuild Alexnet for say  
50 images.

(1000)

out

F10

F9

S8

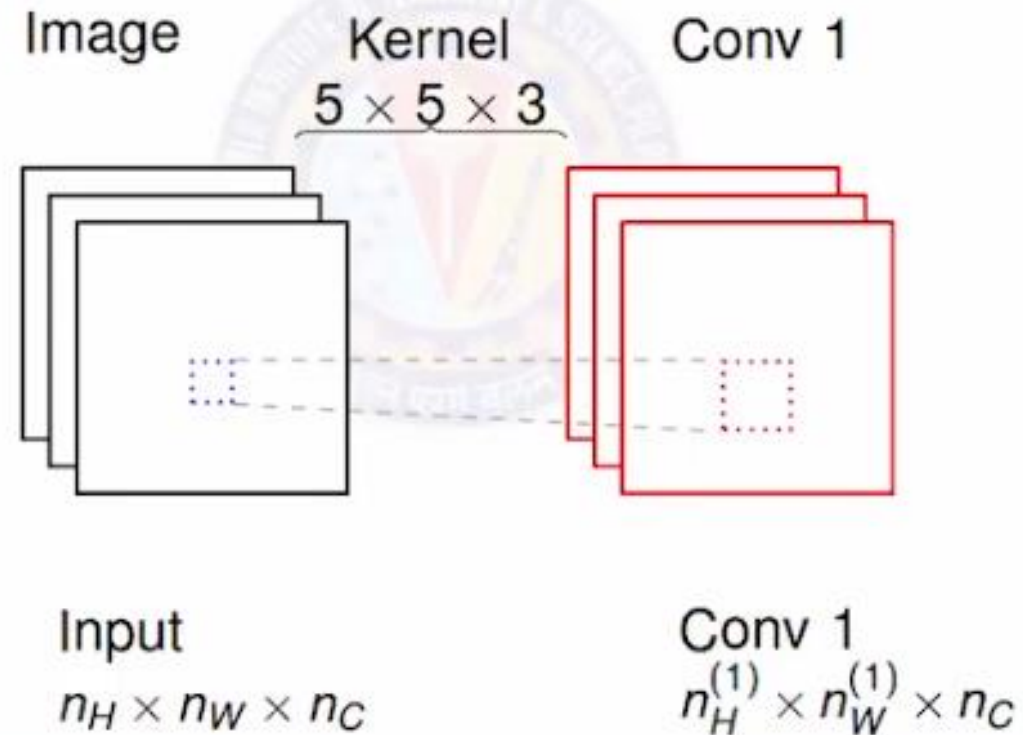
C7

C6  
S5  
S4  
C3  
S2  
C1

In

- Convolution layer

- Perform Convolution operation



- Convolution operation

- Suppose we want to detect edges from an image.
- The image will have vertical edges and horizontal edges.
- Convolve the image with a kernel or filter to extract both the vertical edges and horizontal edges.

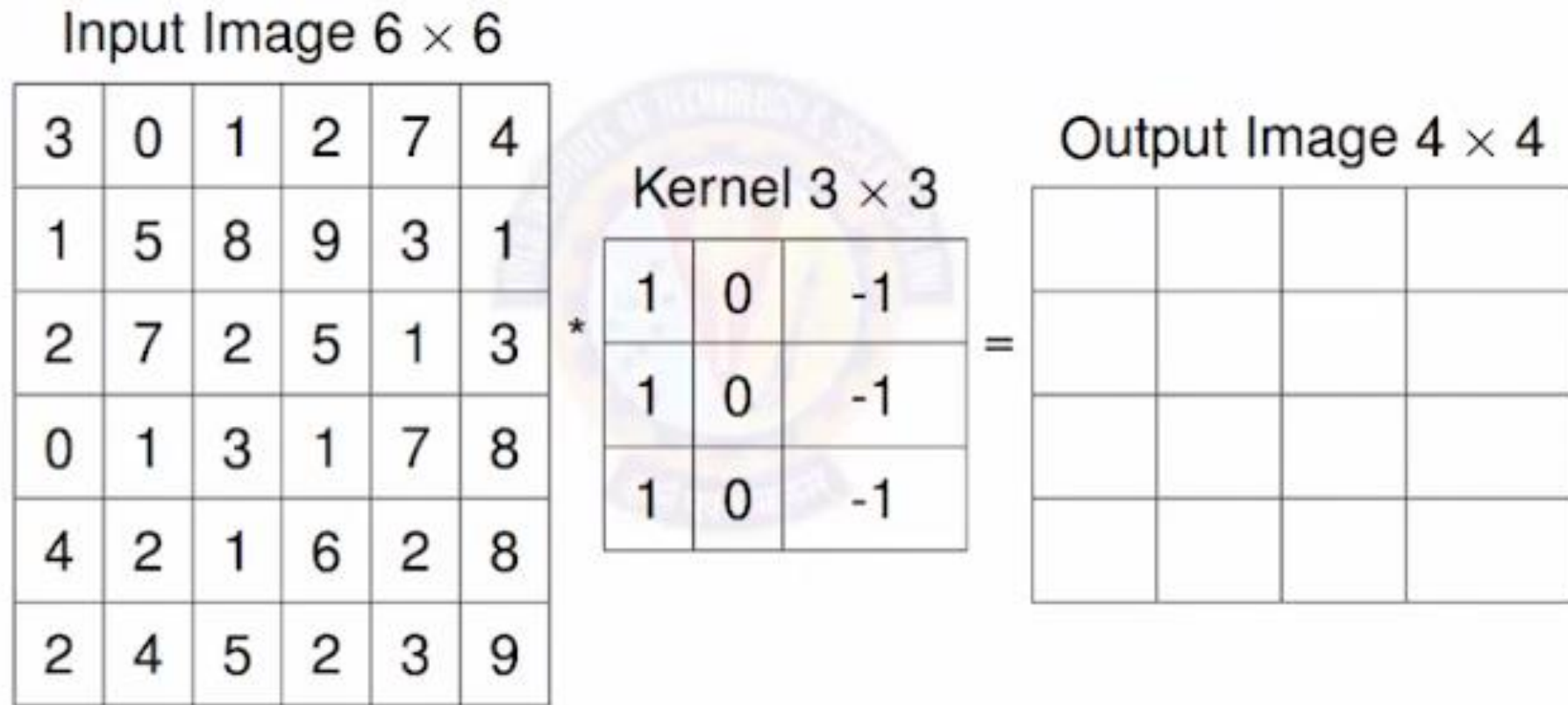
Input Size :  $n \times n$

Kernel Size :  $f \times f$

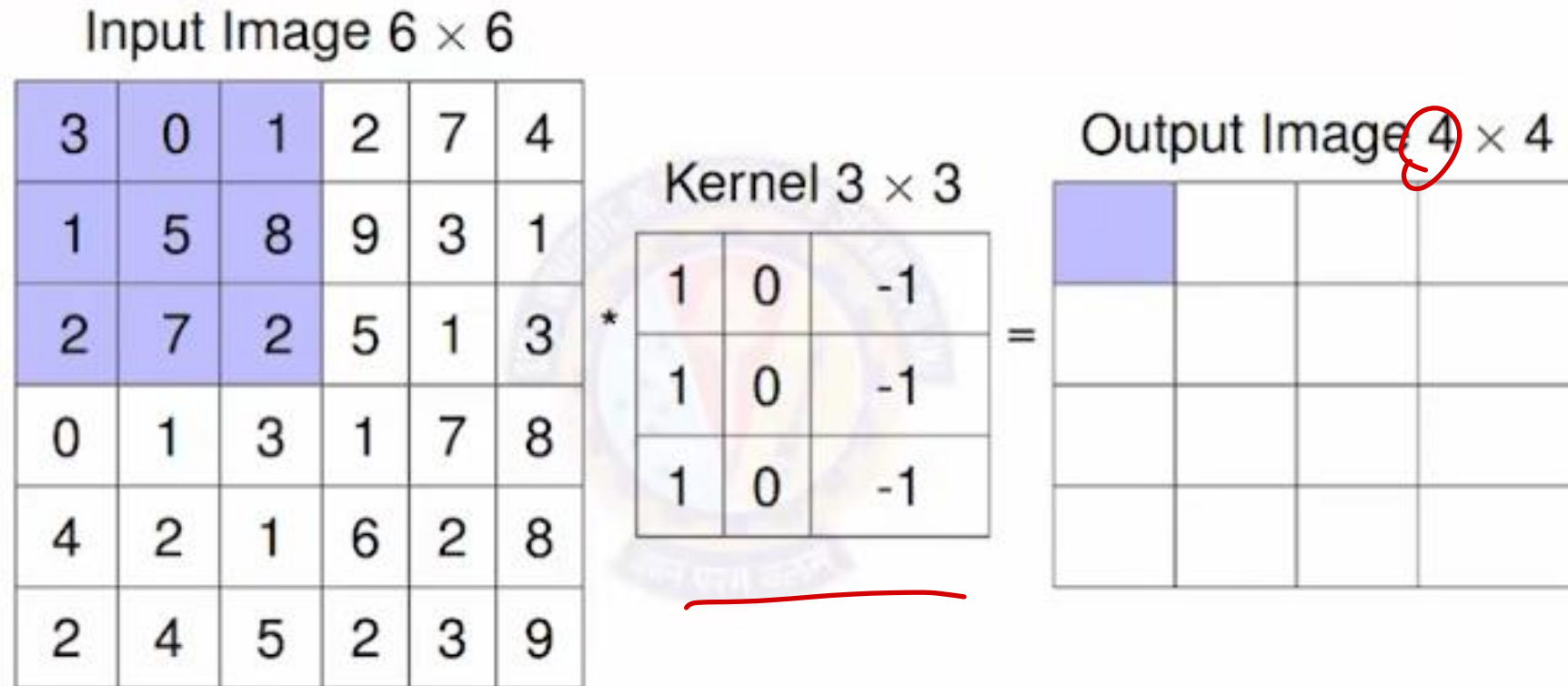
Output Size :  $(n - f + 1) \times (n - f + 1)$



- Convolution operation

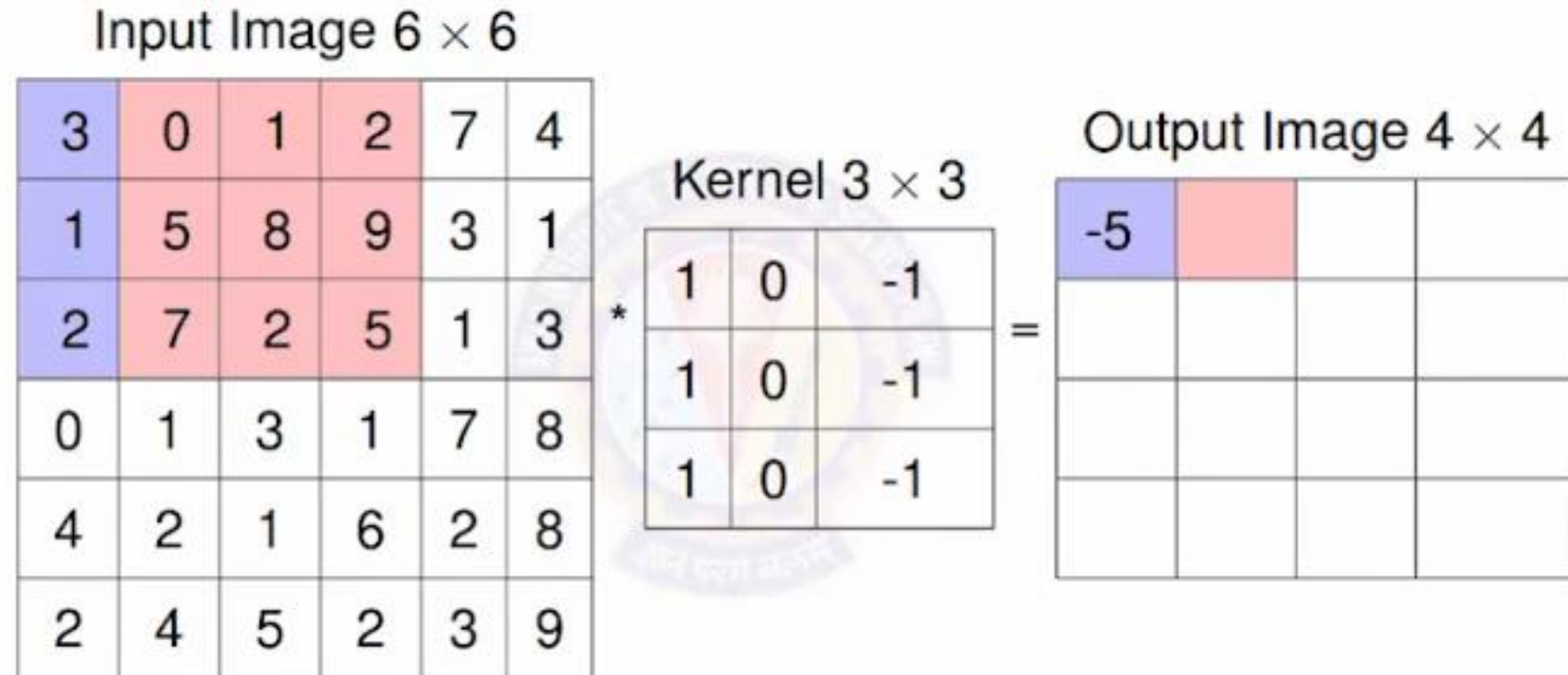


- Convolution operation



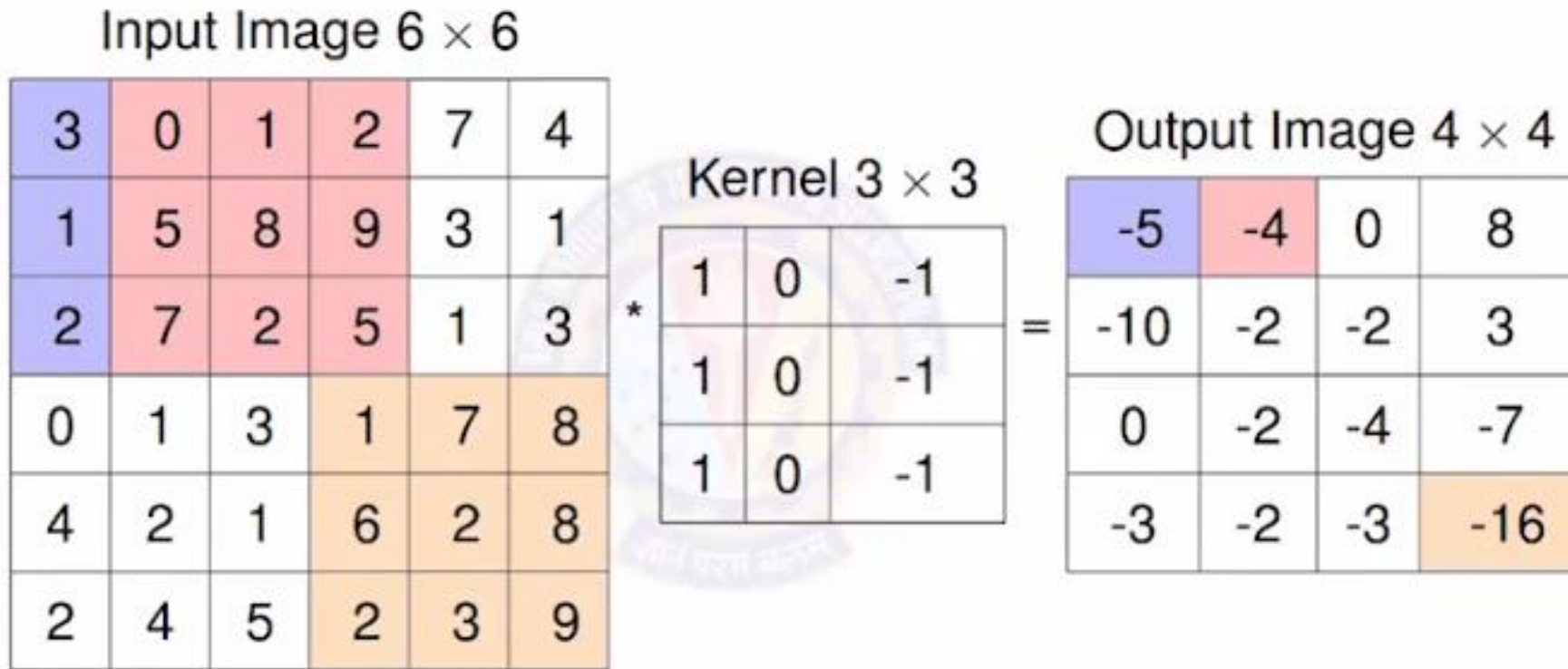
$$\begin{aligned} \text{value} &= 3(1) + 1(1) + 2(1) + 0(0) + 0(5) + 7(0) + 1(-1) + 8(-1) + 2(-1) \\ &= -5 \end{aligned}$$

- Convolution operation



$$\begin{aligned} \text{value} &= 0(1) + 5(1) + 7(1) + 1(0) + 8(0) + 5(0) + 2(-1) + 9(-1) + 5(-1) \\ &= -4 \end{aligned}$$

- Convolution operation



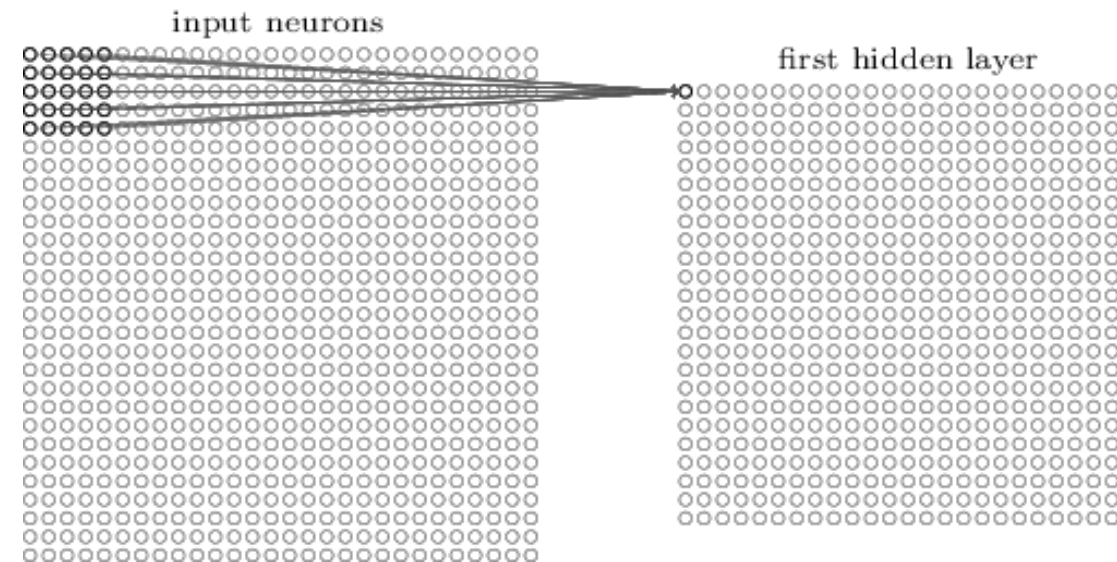
$$\begin{aligned} \text{value} &= 1(1) + 6(1) + 2(1) + 7(0) + 2(5) + 3(0) + 8(-1) + 8(-1) + 9(-1) \\ &= -16 \end{aligned}$$

- Shared weights (and biases)



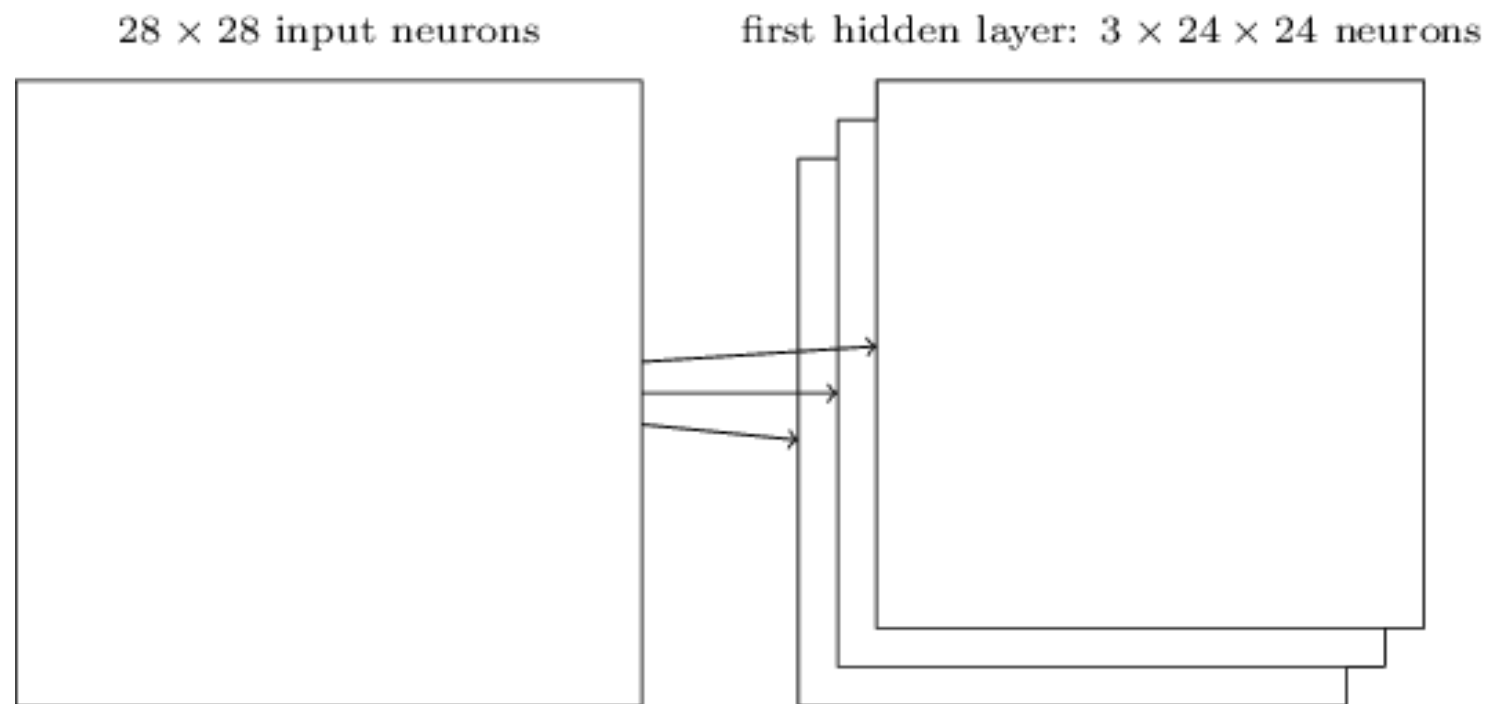
# Shared weights

- We noted in our example that each hidden neuron has a bias and  $5 \times 5$  weights connected to its local receptive field.
- Furthermore: we use the same weights and bias for each of the hidden neurons.
- **What does this imply?**
- All the neurons in the first hidden layer detect exactly the same *feature!* (just at different locations in the input image)



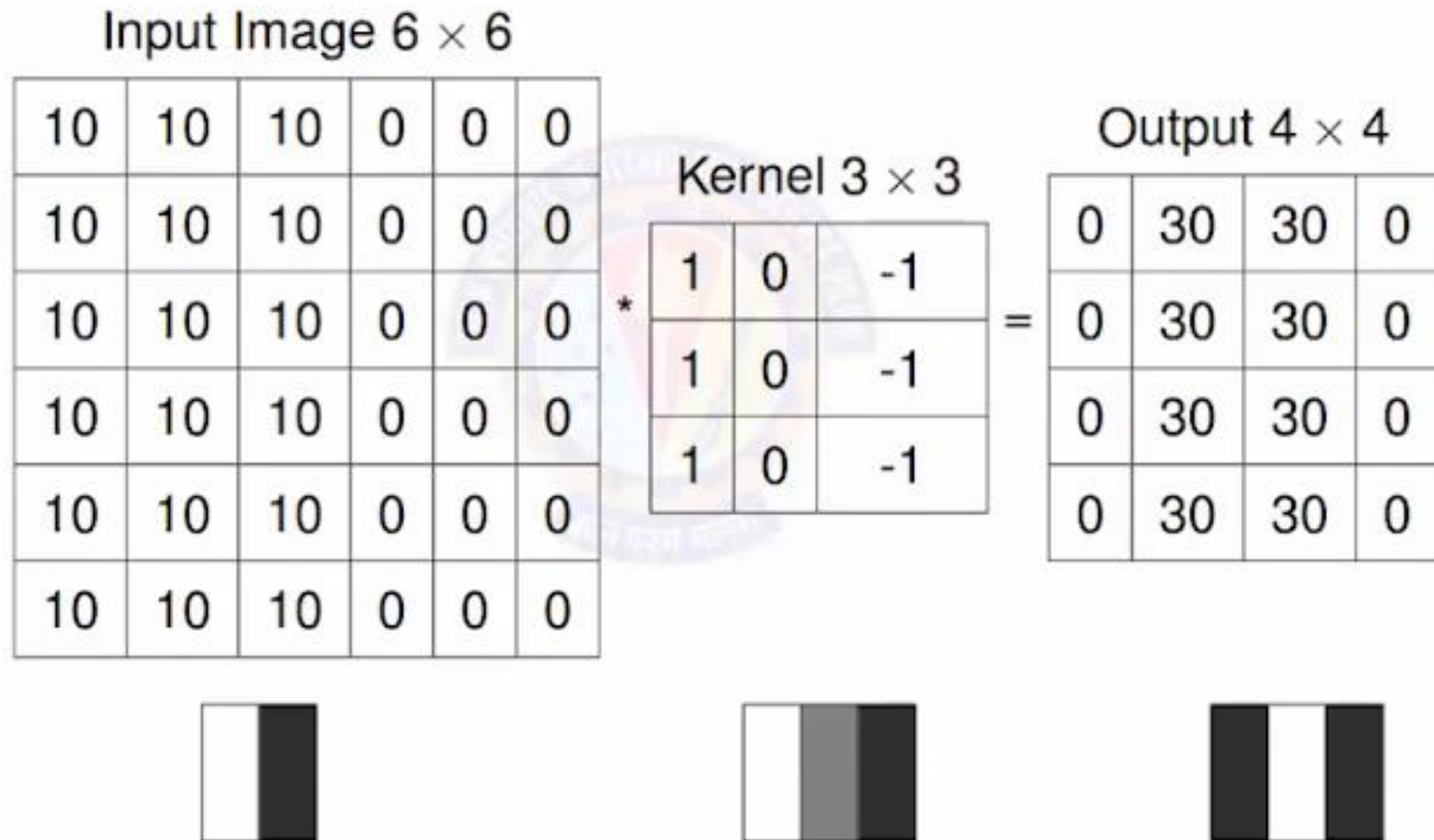
- We often refer the map from the input layer to the hidden layer a *feature map*.
- The shared weights and bias are often said to define a *kernel* or *filter*.

# Simple example



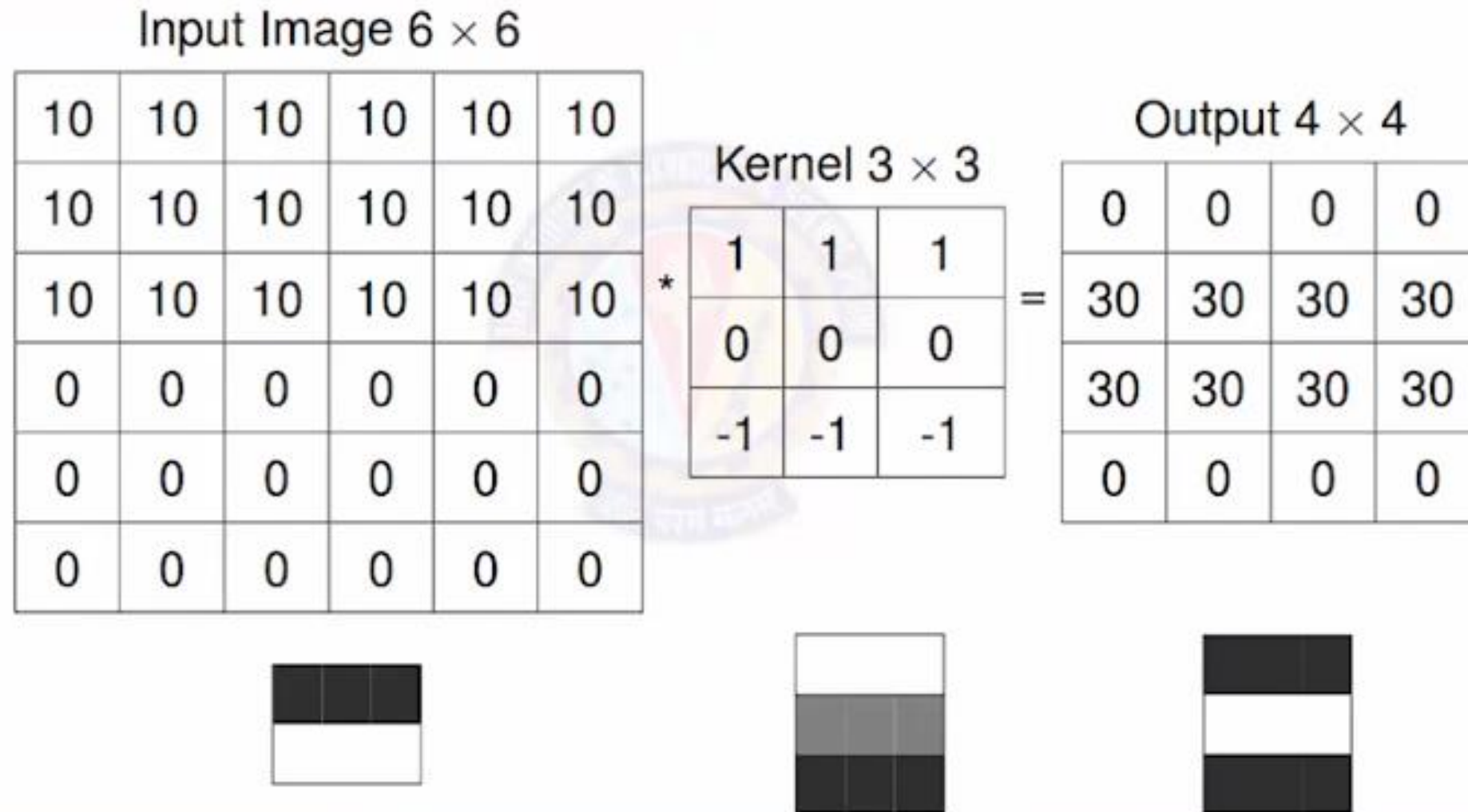
- CNNs as edge detectors

- Vertical edge detector

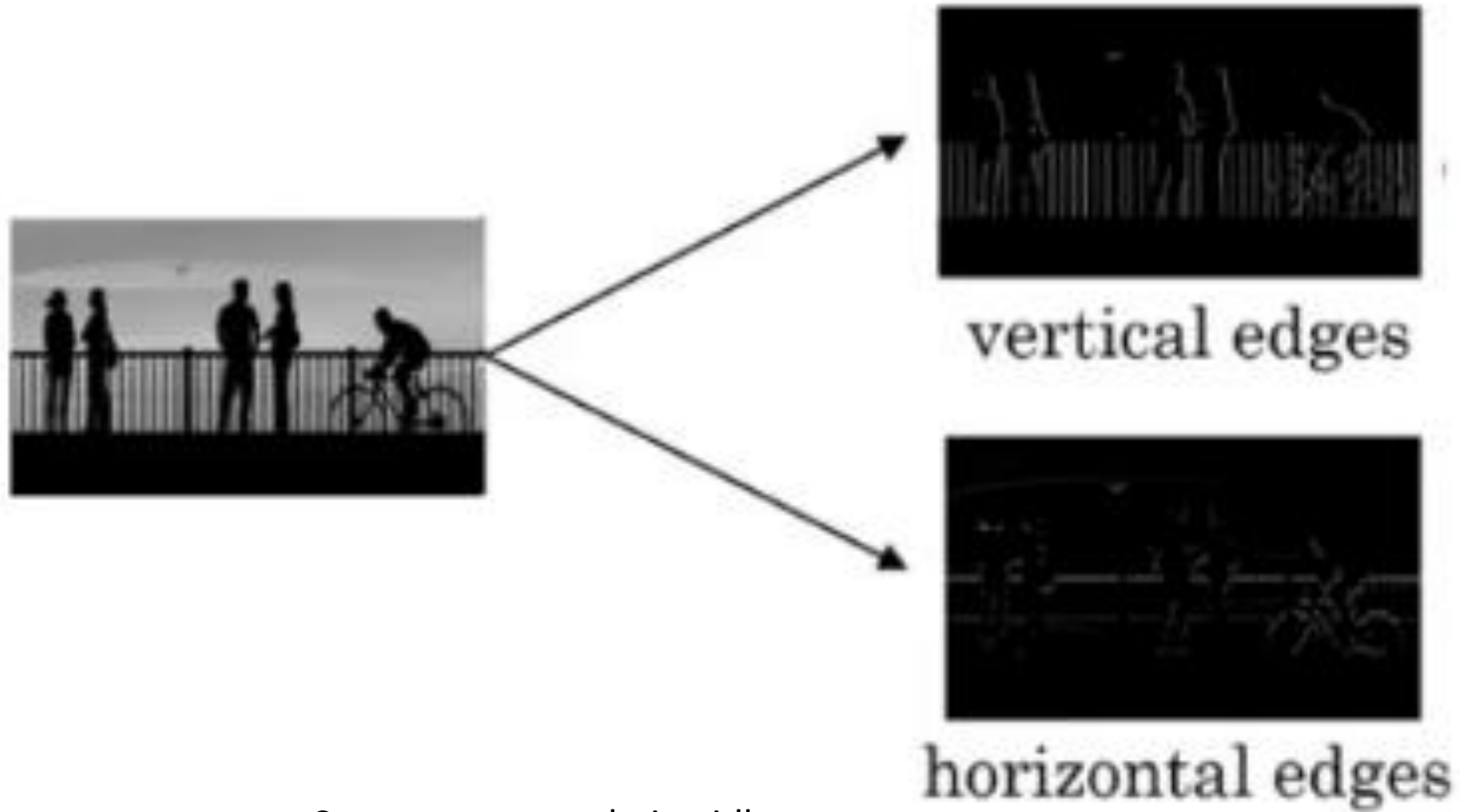




- Horizontal edge detector



- Sample image



Source: [www.analyticsvidhya.com](http://www.analyticsvidhya.com)

- Edge detectors

- Similarly edge at  $45^\circ$ ,  $60^\circ$  and all can be detected, by appropriately choosing the kernels or filters.
- Then stack the different filters to obtain edges.
- The aim of the Convolution layer will be learn the filter or kernel matrix.
- The NN will learn the 9 weights in the filter.

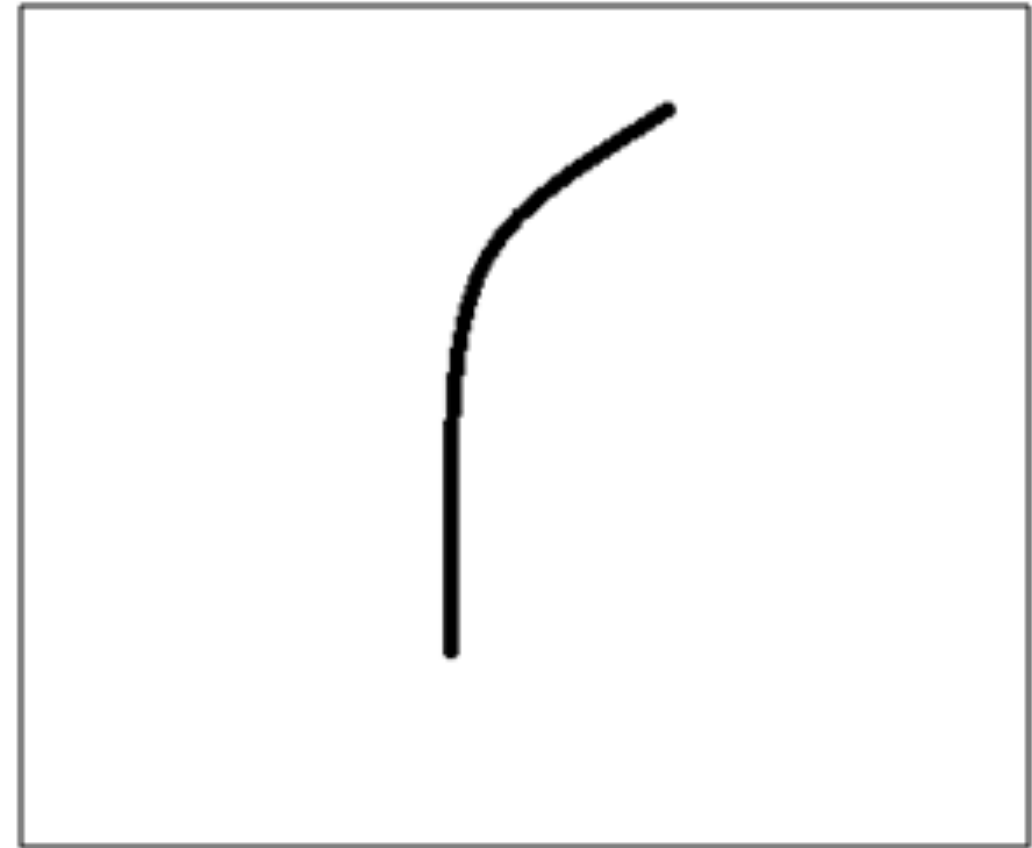
Filter  $3 \times 3$

|       |       |       |
|-------|-------|-------|
| $w_1$ | $w_2$ | $w_3$ |
| $w_4$ | $w_5$ | $w_6$ |
| $w_7$ | $w_8$ | $w_9$ |

- Intuitively...

|   |   |   |    |    |    |   |
|---|---|---|----|----|----|---|
| 0 | 0 | 0 | 0  | 0  | 30 | 0 |
| 0 | 0 | 0 | 0  | 30 | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 0  | 0  | 0  | 0 |

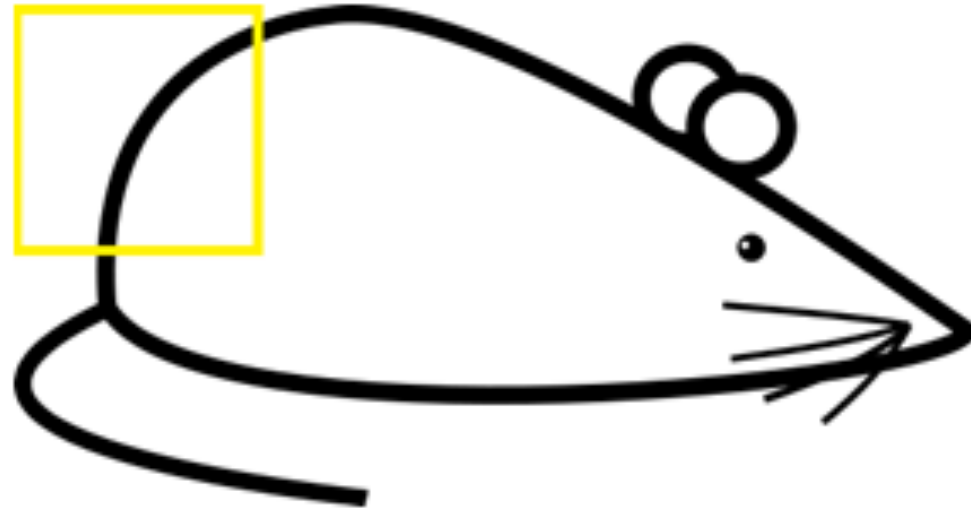
Pixel representation of filter



Visualization of a curve detector filter



Original image



Visualization of the filter on the image

*Curve Detection*



Visualization of the receptive field

|   |   |   |    |    |    |    |
|---|---|---|----|----|----|----|
| 0 | 0 | 0 | 0  | 0  | 0  | 30 |
| 0 | 0 | 0 | 0  | 50 | 50 | 50 |
| 0 | 0 | 0 | 20 | 50 | 0  | 0  |
| 0 | 0 | 0 | 50 | 50 | 0  | 0  |
| 0 | 0 | 0 | 50 | 50 | 0  | 0  |
| 0 | 0 | 0 | 50 | 50 | 0  | 0  |
| 0 | 0 | 0 | 50 | 50 | 0  | 0  |

Pixel representation of the receptive field

\*

|   |   |   |    |    |    |   |
|---|---|---|----|----|----|---|
| 0 | 0 | 0 | 0  | 0  | 30 | 0 |
| 0 | 0 | 0 | 0  | 30 | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 0  | 0  | 0  | 0 |

Pixel representation of filter

Multiplication and Summation =  $(50 \times 30) + (50 \times 30) + (50 \times 30) + (20 \times 30) + (50 \times 30) \in 6600$  (A large number!)



Visualization of the filter on the image

|    |    |    |    |   |   |   |
|----|----|----|----|---|---|---|
| 0  | 0  | 0  | 0  | 0 | 0 | 0 |
| 0  | 40 | 0  | 0  | 0 | 0 | 0 |
| 40 | 0  | 40 | 0  | 0 | 0 | 0 |
| 40 | 20 | 0  | 0  | 0 | 0 | 0 |
| 0  | 50 | 0  | 0  | 0 | 0 | 0 |
| 0  | 0  | 50 | 0  | 0 | 0 | 0 |
| 25 | 25 | 0  | 50 | 0 | 0 | 0 |

Pixel representation of receptive field

\*

|   |   |   |    |    |    |   |
|---|---|---|----|----|----|---|
| 0 | 0 | 0 | 0  | 0  | 30 | 0 |
| 0 | 0 | 0 | 0  | 30 | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 30 | 0  | 0  | 0 |
| 0 | 0 | 0 | 0  | 0  | 0  | 0 |

Pixel representation of filter

Multiplication and Summation = 0



- Padding

- Convolution operation - issues

Input Image  $6 \times 6$

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 7 | 4 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

Kernel  $3 \times 3$

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

Output Image  $4 \times 4$

|     |    |    |     |
|-----|----|----|-----|
| -5  | -4 | 0  | 8   |
| -10 | -2 | -2 | 3   |
| 0   | -2 | -4 | -7  |
| -3  | -2 | -3 | -16 |

- Padding

- When filter is convolved with the input image
  - ▶ Image shrinks –  $6 \times 6$  image became a  $4 \times 4$  image.
  - ▶ Pixels in the corners are used only once, when compared the pixels in the middle. The data in the corners are thrown away.
- To resolve, apply padding to the input image before convolution.
- Padding means append zeros around the boundary of the input.

- Convolution with padding

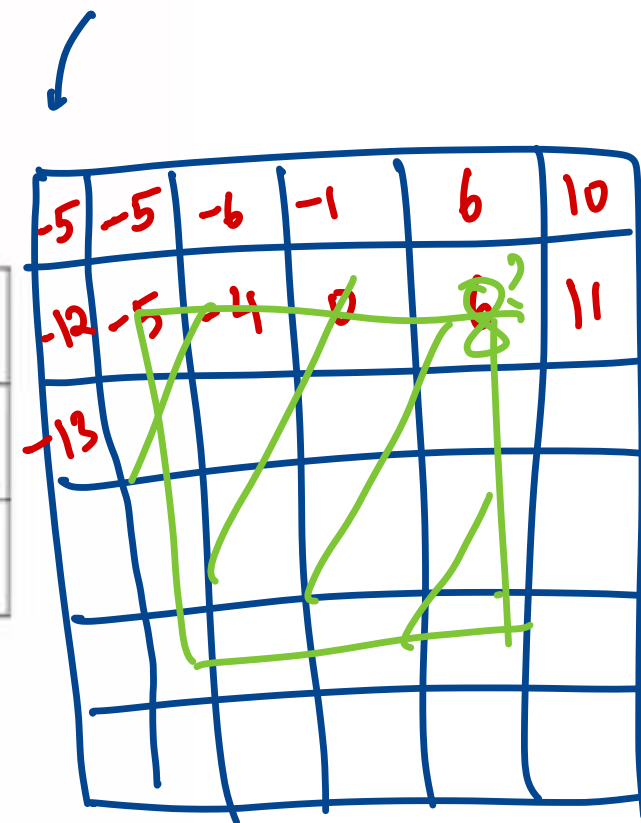
Input Image with padding  $8 \times 8$

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 3 | 0 | 1 | 2 | 7 | 4 | 0 |
| 0 | 1 | 5 | 8 | 9 | 3 | 1 | 0 |
| 0 | 2 | 7 | 2 | 5 | 1 | 3 | 0 |
| 0 | 0 | 1 | 3 | 1 | 7 | 8 | 0 |
| 0 | 4 | 2 | 1 | 6 | 2 | 8 | 0 |
| 0 | 2 | 4 | 5 | 2 | 3 | 9 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Kernel  $3 \times 3$

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

\*



- Padding

- How much to pad?

to get output  
input as n x n

$$\text{Padding : } p = \frac{f - 1}{2}$$

$$\text{Kernel } 3 \times 3 : p = 1$$

$$\text{Kernel } 5 \times 5 : p = 2$$

$$\text{Kernel } 7 \times 7 : p = 3$$

- Padding preserves the input size.

Input Size :  $n \times n$

Padding :  $p$

Kernel Size :  $f \times f$

Output Size :  $(n + 2p - f + 1) \times (n + 2p - f + 1)$

$\hookrightarrow$

$\hookrightarrow$

- Padding

Eg. `Keras.LayerConv2D(filters=32, kernel-size=7, padding='same')` 

- Two kinds of padding

- ▶ Valid Padding

- ★ Padding is not applied.

$$p = 0$$

- ★ Input size  $\neq$  output size.

- ★ Output shrinks when compared to the input.

- ▶ Same Padding

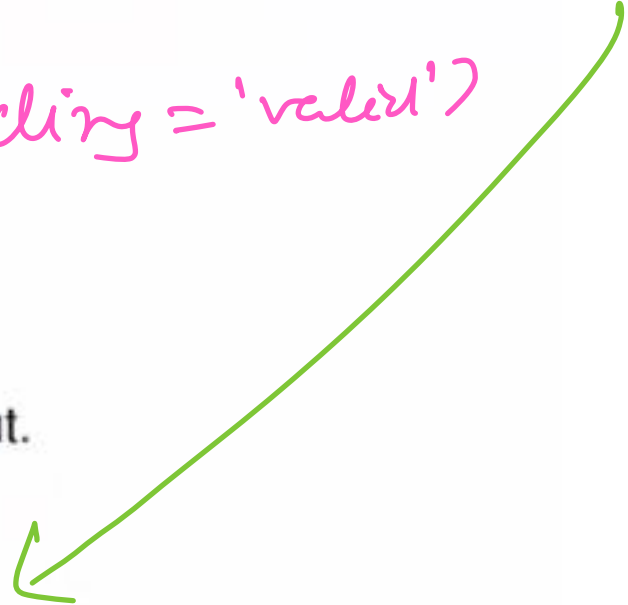
- ★ Padding is applied.

$$p = \frac{f - 1}{2}$$

- ★ Input size = output size.

- ★ We maintain the size of input and output as the same.

padding = 'valid'





input  $n \times n$

output

$$\left[ \frac{n - f + s}{s} \right]$$

input image =  $n \times n$   
filter =  $f \times f$   
stride =  $s$

$n \times m$

$n \times m$

$\times \left[ \frac{n - f + s}{s} \right]$

• Striding

image  $(n \times m)$

padding = mask!

- Striding

- Skip by how many pixels.

Input Size :  $n \times n$

Padding :  $p$

Stride :  $s$

Kernel Size :  $f \times f$

Output Size :  $\left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$

$$p = \left( \frac{f-1}{2} \right)$$

- The filter must lie completely inside the input.

Input Image  $7 \times 7$

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 4 | 7 | 4 | 6 | 2 | 9 |
| 6 | 6 | 9 | 8 | 7 | 4 | 3 |
| 3 | 4 | 8 | 3 | 8 | 9 | 7 |
| 7 | 8 | 3 | 6 | 6 | 3 | 4 |
| 4 | 2 | 1 | 8 | 2 | 4 | 6 |
| 3 | 2 | 4 | 1 | 9 | 8 | 3 |
| 0 | 1 | 3 | 9 | 2 | 1 | 4 |

Kernel  $3 \times 3$

Stride = 2

|   |    |   |   |
|---|----|---|---|
| * | 3  | 4 | 4 |
|   | 1  | 0 | 2 |
|   | -1 | 0 | 3 |

Output  $3 \times 3$

|    |  |  |
|----|--|--|
| 91 |  |  |
|    |  |  |
|    |  |  |

Input Image  $7 \times 7$

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 4 | 7 | 4 | 6 | 2 | 9 |
| 6 | 6 | 9 | 8 | 7 | 4 | 3 |
| 3 | 4 | 8 | 3 | 8 | 9 | 7 |
| 7 | 8 | 3 | 6 | 6 | 3 | 4 |
| 4 | 2 | 1 | 8 | 2 | 4 | 6 |
| 3 | 2 | 4 | 1 | 9 | 8 | 3 |
| 0 | 1 | 3 | 9 | 2 | 1 | 4 |

Kernel  $3 \times 3$

Stride = 2

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

Output  $3 \times 3$

|    |     |  |
|----|-----|--|
| 91 | 100 |  |
|    |     |  |
|    |     |  |

Input Image  $7 \times 7$

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 4 | 7 | 4 | 6 | 2 | 9 |
| 6 | 6 | 9 | 8 | 7 | 4 | 3 |
| 3 | 4 | 8 | 3 | 8 | 9 | 7 |
| 7 | 8 | 3 | 6 | 6 | 3 | 4 |
| 4 | 2 | 1 | 8 | 2 | 4 | 6 |
| 3 | 2 | 4 | 1 | 9 | 8 | 3 |
| 0 | 1 | 3 | 9 | 2 | 1 | 4 |

Kernel  $3 \times 3$

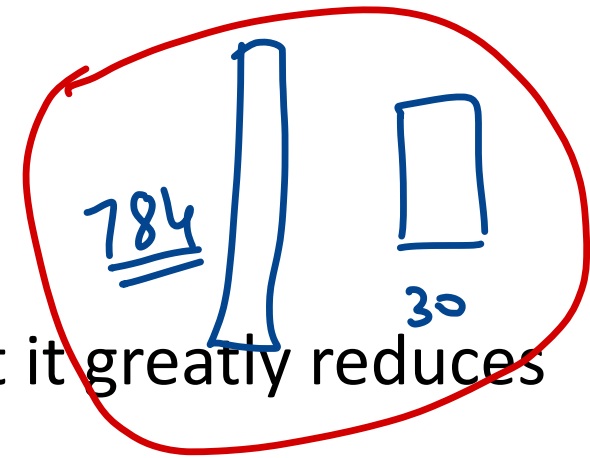
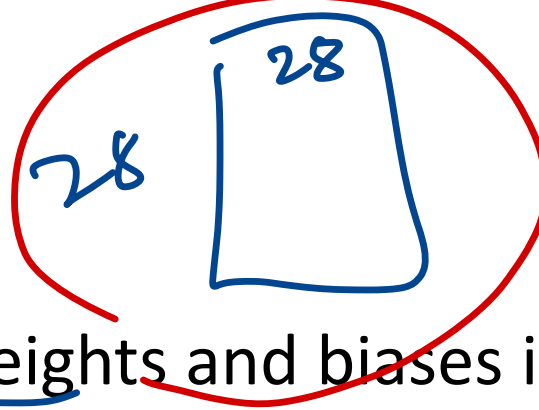
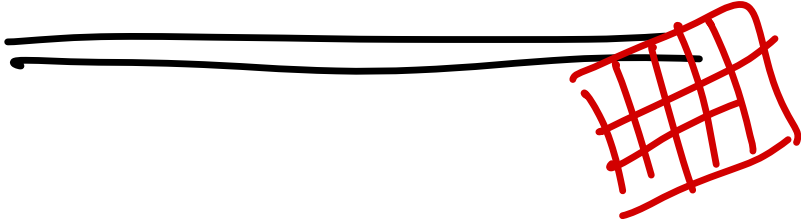
Stride = 2

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

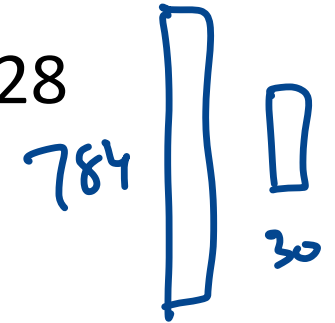
Output  $3 \times 3$

|    |     |     |
|----|-----|-----|
| 91 | 100 | 83  |
| 69 | 91  | 127 |
| 44 | 72  | 74  |

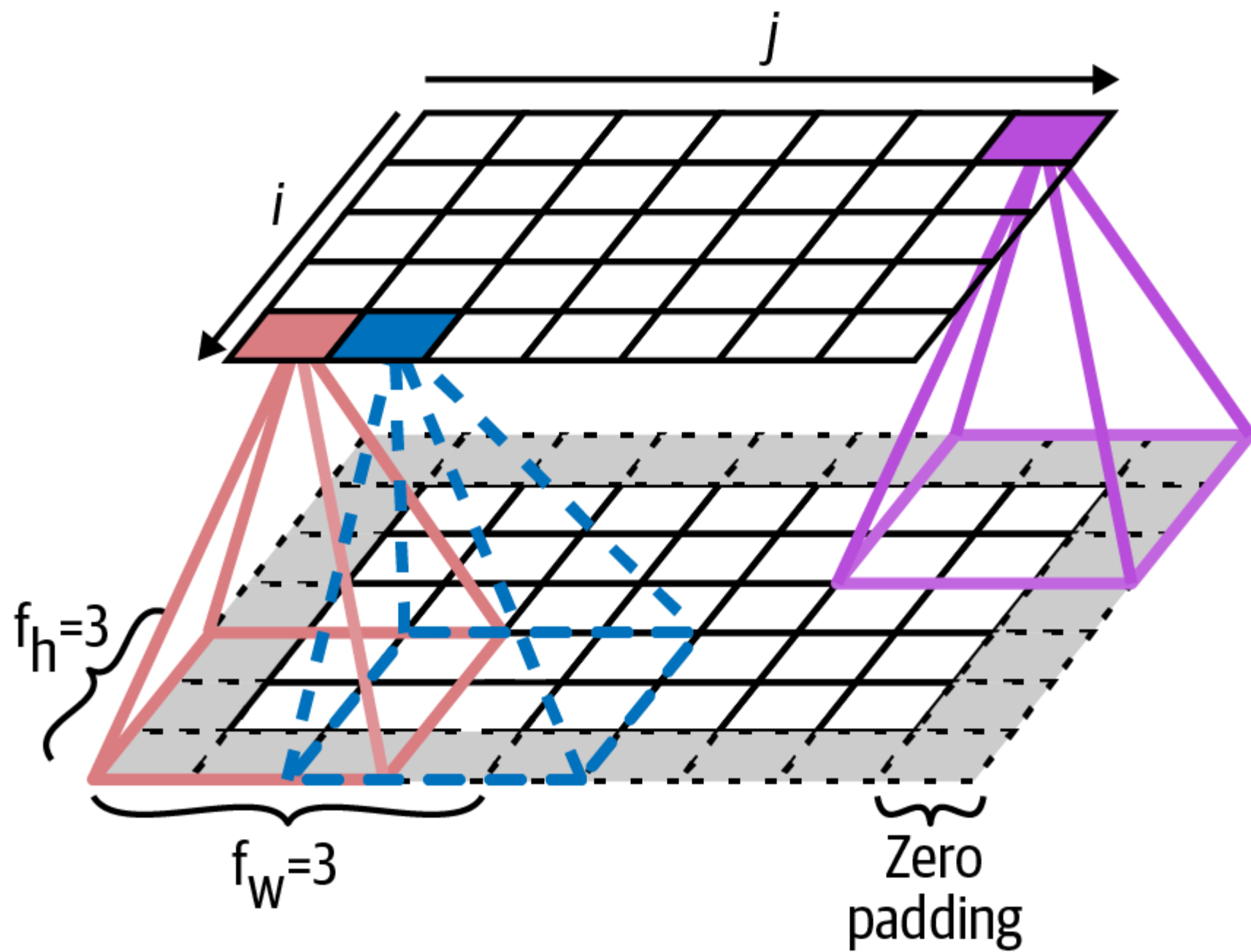
- Parameter size

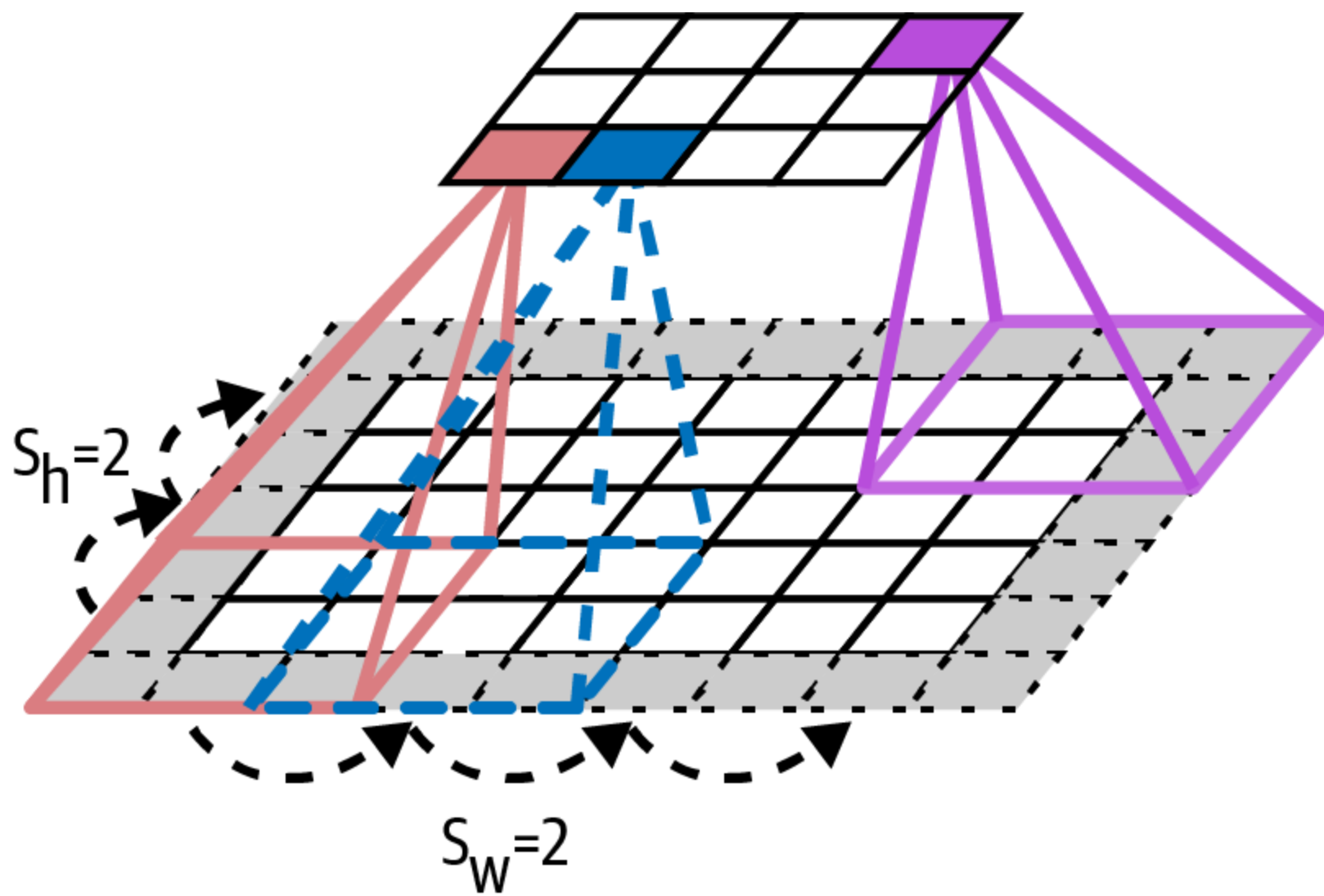


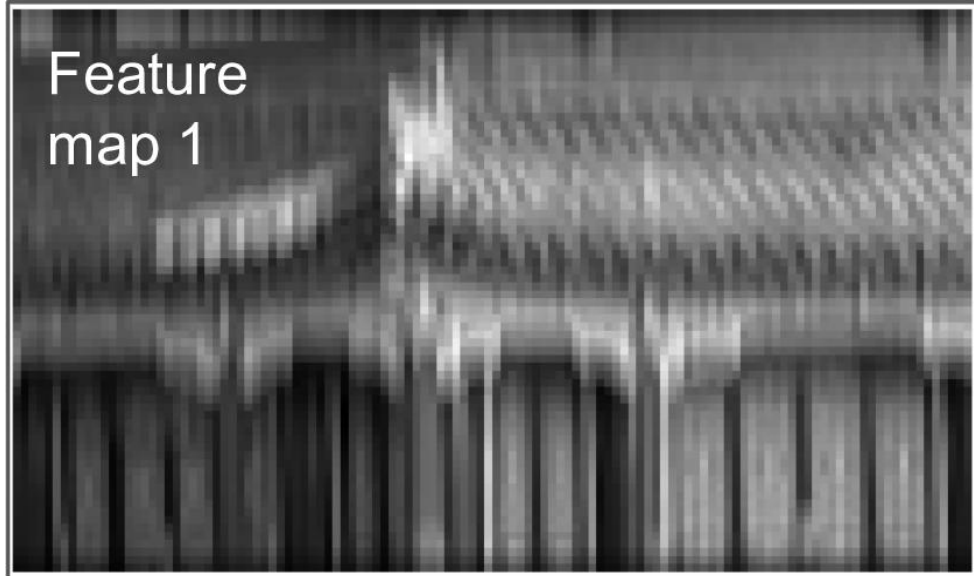
- A big advantage of sharing weights and biases is that it greatly reduces the number of parameters involved.
- For each feature map, we need  $25 = 5 \times 5$  shared weights, plus a single shared bias. So each feature map requires 26 parameters.
- For 20 feature maps  $\rightarrow 20 \times 26 = 520$  parameters for the convolutional layer.
- By comparison: suppose a fully connected first layer with  $784 = 28 \times 28$  input neurons, and a relatively modest 30 hidden neurons.
- That's  $784 \times 30$  weights + 30 biases, total 23,550 parameters.
- The fully-connected layer has more than 40 times as many parameters as the convolutional layer.









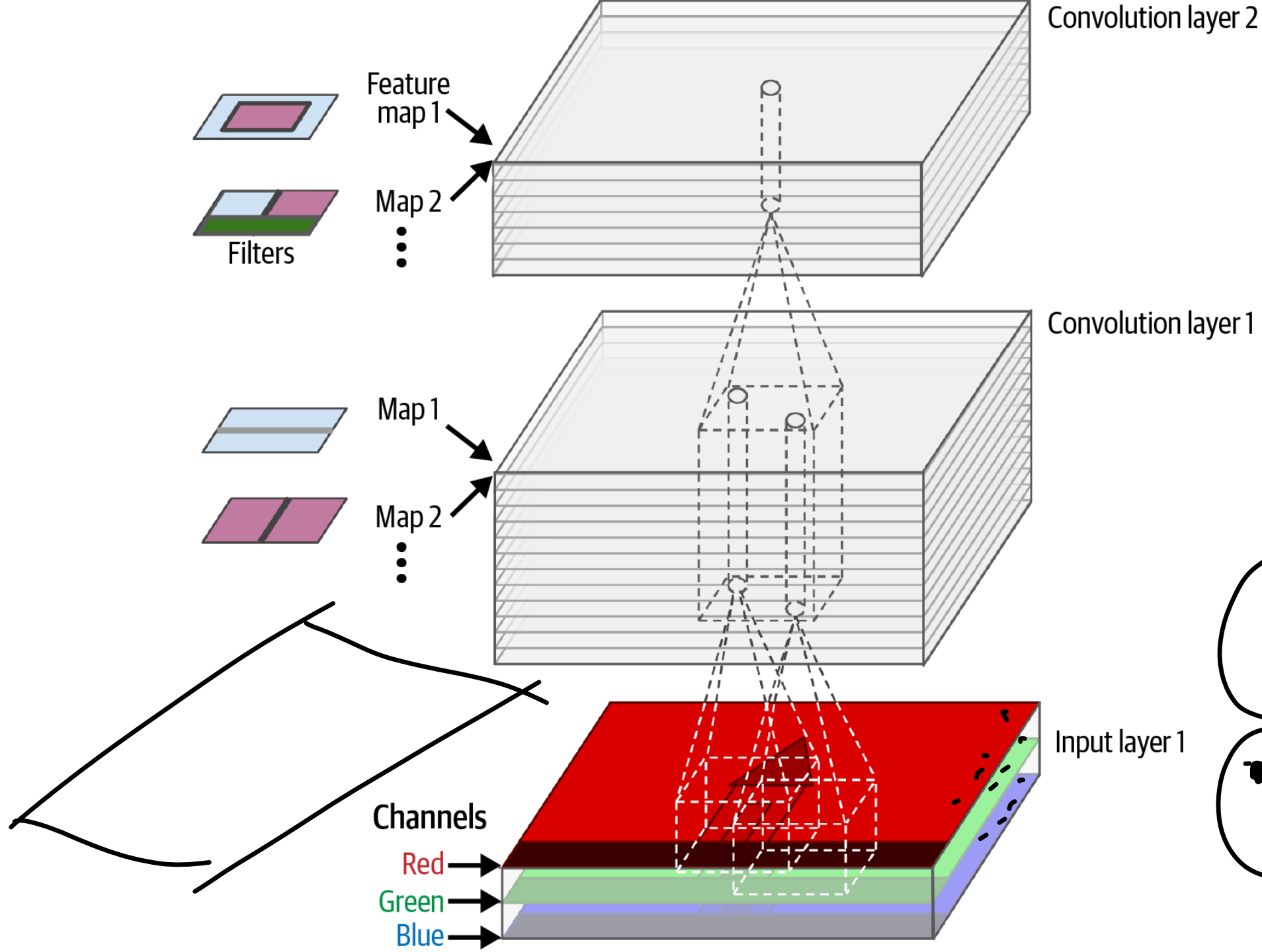


Vertical filter



Horizontal filter





- Pooling layer

- Pooling layer
- Pooling layer is used for subsampling.
- It reduces the size of the representation.
- It also speeds up the computation.
- Pooling is applied on each of the channels
- It is a fixed computation
- No use of gradient descent or activation function.
- Padding is not used
- The output dimensions are:
  - $\left[ \frac{n-f}{s} + 1 \right] \times \left[ \frac{n-f}{s} + 1 \right]$



- Pooling
- Broadly two types:
- Max pooling
- Average pooling

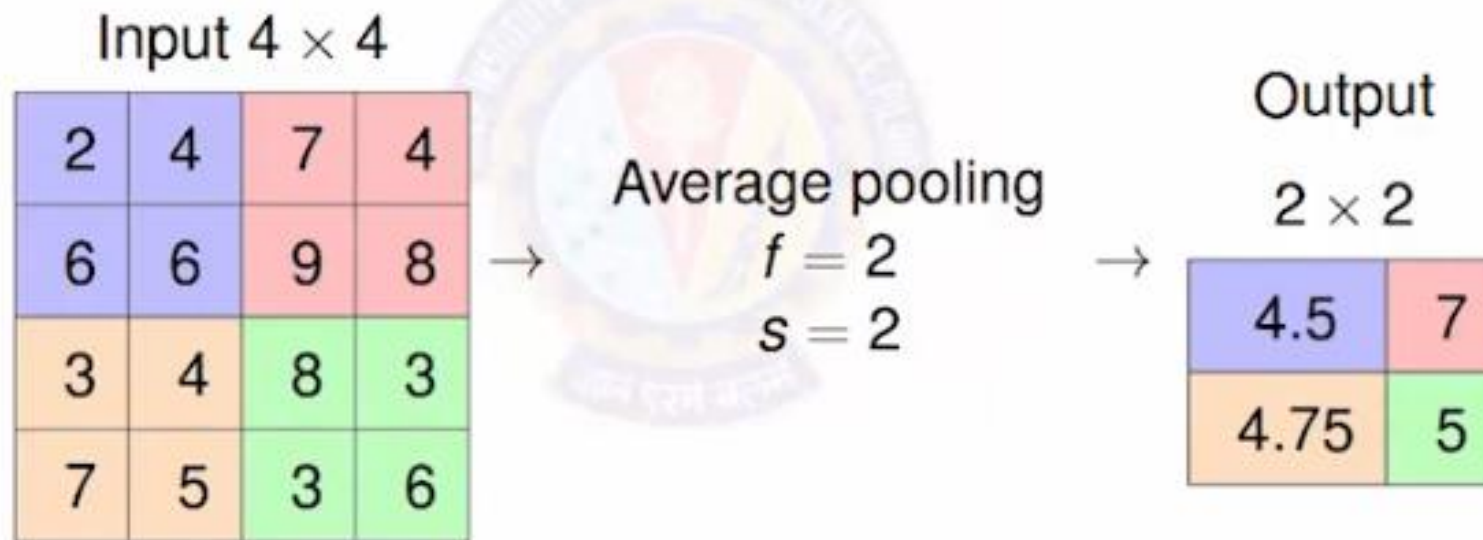
- Max pooling

- Take the maximum of the sub region that is considered.



- Average pooling

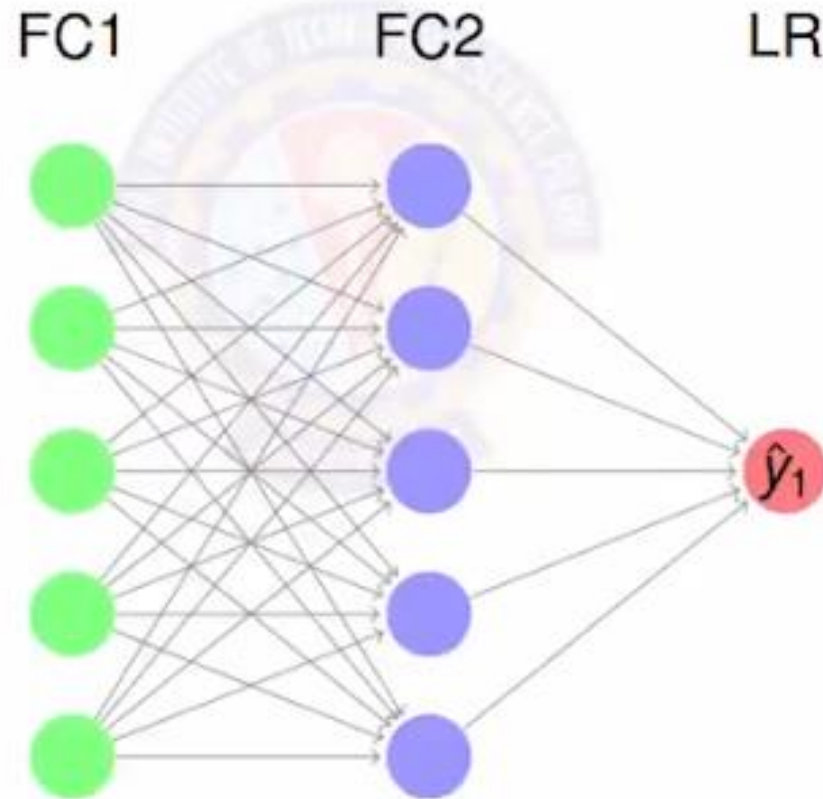
- Compute the average of the sub region that is considered.



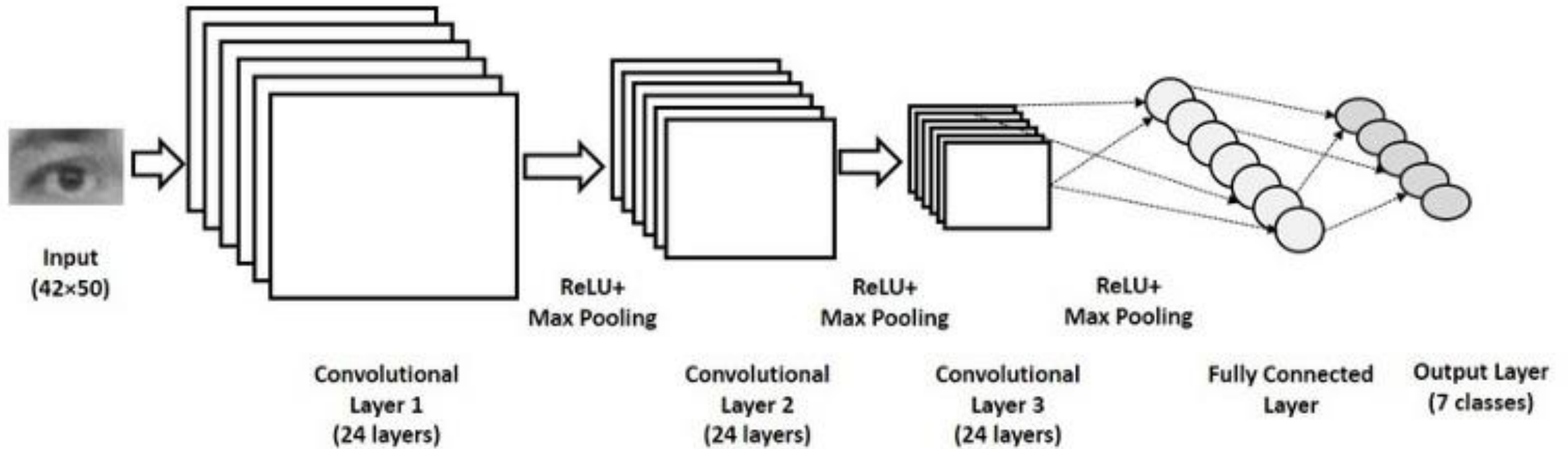
- Flattening

- Converting the data into a 1-dimensional array for inputting it to the next layer.
- We **flatten** the output of the convolutional layers to create a single long feature vector.
- This is connected to the final classification model, which is a ***fully-connected*** layer.

- Fully connected layer
- Stack of neurons in one layer connected to every other neuron in the next layer



- Putting the pieces together



Source: <https://towardsdatascience.com>

- Why CNNs?
- **Feature learning!**
- **Shared weights!**
- CNN take advantage of **local spatial coherence** of images, dramatically reducing the number of operation needed to process an image.
- CNNs are more efficient in terms of **memory and complexity**.